



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS FLORIANÓPOLIS
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIAS DA COMPUTAÇÃO

Lucas Pereira da Silva

**Refatoração Automática de Classes de Teste Usando Métricas de Similaridade
e Algoritmos de Agrupamento**

Florianópolis
2025

Lucas Pereira da Silva

**Refatoração Automática de Classes de Teste Usando Métricas de Similaridade
e Algoritmos de Agrupamento**

Texto submetido ao Programa de Pós-Graduação em Ciências da Computação da
Universidade Federal de Santa Catarina como requisito para o Seminário de
Andamento do Doutorado.

Orientador: Prof. Dr. Ronaldo dos Santos Mello

Coorientadora: Prof.^a Dr.^a Patricia Vilain

Florianópolis

2025



RESUMO

A manutenibilidade do código de teste é crucial durante a atividade de teste. Isso porque uma baixa capacidade de manutenção do código de teste dificulta tanto a adição de novos testes quanto o ajuste dos testes já existentes quando ocorrem mudanças no código de aplicação. Mesmo que o código de teste já possua boa manutenibilidade, ela tende a se degradar com o tempo sem um processo contínuo de refatoração. Assim, é preciso que os testes sejam continuamente refatorados. Uma atividade efetiva de refatoração para melhorar a manutenibilidade consiste em reduzir a duplicação no código de teste. Este trabalho investiga **estratégias automáticas de refatoração de código** aplicadas ao contexto do código de teste. Para isso, investigamos na literatura as abordagens mais recentes sobre **refatoração de código de teste**. Também apresentamos uma nova proposta para refatoração automática de classes de teste. A proposta utiliza métricas de similaridade e algoritmos de agrupamento para redistribuir os testes entre as classes de teste. O objetivo da redistribuição é agrupar, em uma mesma classe, os testes similares entre si visando maximizar o reuso de código de teste através da estratégia de configuração implícita, comumente presente nos *frameworks* de teste. Maximizar o reuso de código de teste de forma automática pode melhorar a manutenibilidade do código de teste sem aumentar o esforço de desenvolvimento.

Palavras-chave: teste de *software*; refatoração; reuso de código; métrica de similaridade; algoritmo de agrupamento, configuração implícita.

LISTA DE FIGURAS

Figura 1. Teste de criação de usuário.	13
Figura 2. Teste de inserção de usuário.....	13
Figura 3. Teste calculadora com dois testes.....	18
Figura 4. Teste com a estratégia configuração local.....	20
Figura 5. Teste com a estratégia configuração implícita.	21
Figura 6. Teste com a estratégia configuração delegada.	22
Figura 7. Diagrama de atividades com as etapas do estudo sistemático.	37
Figura 8. Classe de teste com duplicação de código.....	44
Figura 9. Classe de teste sem duplicação de código.....	45
Figura 10. Teste com duplicação de código em outra classe de teste.	46
Figura 11. Diagrama de atividades do modelo para refatoração automática.....	47
Figura 12. Árvore abstrata de sintaxe de uma classe de teste.	48
Figura 13. Matriz de similaridade de testes.	50
Figura 14. Dois grupos para três testes.....	51
Figura 15. Um grupo para três testes.	51

LISTA DE TABELAS

Tabela 1. Escopo de uso de técnicas de reuso.	31
Tabela 2. Forma de identificação de duplicação de técnicas de reuso.....	32
Tabela 3. Processo de refatoração de técnicas de reuso.	32
Tabela 4. Esforço de refatoração de técnicas de reuso.....	33

SUMÁRIO

1. INTRODUÇÃO	11
1.1. PROBLEMA DE PESQUISA.....	15
1.2. HIPÓTESE DE PESQUISA	15
1.3. OBJETIVOS.....	16
1.4. ORGANIZAÇÃO DO TRABALHO.....	16
2. FUNDAMENTAÇÃO TEÓRICA	17
2.1. TESTE DE SOFTWARE	17
2.1.1. Categorias e estratégias de configuração de acessórios	19
2.1.1.1. <i>Estratégia configuração local</i>	20
2.1.1.2. <i>Estratégia configuração implícita</i>	20
2.1.1.3. <i>Estratégia de configuração delegada</i>	22
2.1.1.4. <i>Categoria de estratégias de configuração de acessórios compartilhados ...</i>	23
2.2. MÉTRICAS DE SIMILARIDADE	23
2.2.1. Métricas de similaridade de código	23
2.2.1.1. <i>Métricas baseadas em medidas do código</i>	24
2.2.1.2. <i>Métricas baseadas em texto</i>	24
2.2.1.3. <i>Métricas baseadas em tokens</i>	25
2.2.1.4. <i>Métricas baseadas em árvores</i>	25
2.2.1.5. <i>Métricas baseadas em grafo</i>	25
3. ESTADO DA ARTE	27
3.1. REVISÃO EXPLORATÓRIA	27
3.1.1. Teste de software e métricas de similaridade	28
3.1.2. Reuso de código de teste	28
3.1.2.1. <i>DbUnit</i>	28
3.1.2.2. <i>Picon</i>	29
3.1.2.3. <i>Estória</i>	29

3.1.3. Comparação entre os trabalhos.....	30
3.1.3.1. <i>Escopo</i>	30
3.1.3.2. <i>Forma de identificação de duplicação</i>	31
3.1.3.3. <i>Processo de refatoração</i>	32
3.1.3.4. <i>Esforço de refatoração</i>	33
3.1.4. Discussão	34
3.2. ESTUDO SISTEMÁTICO	35
4. PROPOSTA	43
5. CONCLUSÃO	53

1. INTRODUÇÃO

Teste de *software* é uma das atividades presentes no processo de desenvolvimento e manutenção de *software*. A percepção sobre sua utilidade tem evoluído nos últimos anos. Teste de *software* não é mais visto como uma atividade a ser realizada apenas ao final da etapa de codificação para detectar falhas. A atividade de teste passou a estar presente durante todo o processo de desenvolvimento e manutenção de *software* (Bourque e Fairley, 2014). Meszaros (2007) vê a atividade de teste como parte intrínseca do processo de desenvolvimento de *software*. Testes devem, portanto, ser incorporados ao processo de desenvolvimento de modo que a evolução do *software* seja acompanhada pela evolução dos testes. Nesse sentido, a atividade de teste de *software* passou a incorporar propósitos variados, como detectar falhas (Tiwari e Goel, 2013), prevenir que erros sejam introduzidos (Beizer, 1990), prover um indicativo de que o *software* funciona (Bertolino, 2007), auxiliar na compreensão do projeto e do código do sistema (Freeman e Pryce, 2009) e servir como um meio para especificação (Alvestad, 2007) e documentação dos requisitos (Haugset e Hanssen, 2008).

Conforme definido por (Bourque e Fairley, 2014), teste de *software* consiste na verificação dinâmica de que um programa provê um comportamento esperado a partir de um conjunto finito de casos de teste, adequadamente selecionados a partir de um domínio de execução potencialmente infinito. Tal atividade pode ser realizada manualmente ou de forma automatizada através da criação de *scripts* de teste. A criação de *scripts* de teste, por sua vez, pode ser realizada através de diferentes abordagens das quais destacam-se a codificação manual, a utilização de ferramentas que geram os *scripts* de teste a partir da gravação de interações manuais com o sistema em teste (*System Under Test* – SUT) e a geração baseada em modelos na qual os *scripts* de testes são gerados automaticamente a partir dos requisitos (Dalal *et al.*, 1999).

Este trabalho se enquadra no contexto de testes automatizados codificados manualmente. Nesse contexto, a qualidade do código de teste tem especial importância, uma vez que, assim como o código da aplicação, o código de teste também precisará ser mantido, compreendido e ajustado durante todo o projeto (Greiler *et al.*, 2013). Segundo Tsai *et al.* (2003), a atividade de teste pode consumir de 50% a 60% de todo o esforço de desenvolvimento e manutenção de *software*.

Para Meszaros (2007), a atividade de teste não deveria aumentar o esforço total de desenvolvimento e manutenção do *software*, uma vez que o esforço gasto com essa atividade deve ser compensado por uma redução no esforço total devido à garantia de melhoria da qualidade do *software*.

Segundo Emery (2009), o motivo comum pelo qual pessoas e organizações abandonam testes automatizados está no fato de que os testes se tornam frágeis e com alto custo de manutenção. De acordo com Berner, Weber e Keller (2005), a manutenção dos testes tende a ter um impacto maior no esforço total do projeto se comparado com a implementação inicial dos testes. Pequenas mudanças na implementação do *software* podem afetar uma grande quantidade de testes, e consertá-los demanda grande esforço. Não se pode impedir que os requisitos e a implementação do *software* mudem. Por isso, a melhor forma para manter os custos de manutenção dos testes baixos é tornar os testes mais adaptáveis a mudanças no *software*. Para o autor, um dos fatores chaves para isso é evitar duplicações entre os testes. Segundo Greiler, van Deursen e Storey (2013), o sucesso a longo prazo da automação de testes é fortemente influenciado pela manutenibilidade do código de teste. Para que possa existir uma boa manutenibilidade, os métodos de teste devem ser estruturados claramente, ter nomes representativos, ser pequenos e, sobretudo, deve-se evitar a duplicação de código.

A Figura 1 apresenta um teste onde é realizada a criação de um usuário do sistema, enquanto a Figura 2 apresenta um teste para a inserção de um novo usuário no sistema. Os testes apresentados foram codificados na linguagem Kotlin e utilizam o *framework* de testes JUnit. É possível observar que as linhas 3 a 5 aparecem em ambos os testes. Considerando o conjunto de todos os testes, as linhas 4 a 6 são percebidas como uma duplicação do código de teste. Esse tipo de duplicação de código causa prejuízos à manutenibilidade, uma vez que mudanças no construtor ou nos métodos envolvidos afetam diretamente os dois testes. Embora no exemplo apresentado a duplicação não pareça ter um impacto significativo na manutenibilidade, tal impacto pode crescer tanto quanto maior for a quantidade de testes. De acordo com Berner, Weber e Keller (2005), testes tendem a ser repetitivos e apresentam um alto potencial de reuso. Conforme o *software* evolui e o número de testes aumenta é comum que a quantidade de código duplicado através dos testes também aumente.

Figura 1. Teste de criação de usuário.

```

1 class TesteCriarUsuario {
2     @Test
3     fun criarUsuario() {
4         val joao = Usuario()
5         joao.fixarNome("João da Silva")
6         joao.fixarProfissao("Programador")
7         assertNull(joao.obterIdentificador())
8         assertEquals("João da Silva", joao.obterNome())
9         assertEquals("Programador", joao.obterProfissao())
10    }
11 }

```

Figura 2. Teste de inserção de usuário.

```

1 class TesteInserirUsuario {
2     @Test
3     fun inserirUsuario() {
4         val joao = Usuario()
5         joao.fixarNome("João da Silva")
6         joao.fixarProfissao("Programador")
7         val repositorioDeUsuario = RepositorioDeUsuario()
8         val identificador = repositorioDeUsuario.inserir(joao)
9         assertEquals(identificador, joao.obterIdentificador())
10    }
11 }

```

Uma prática frequente de reuso de código de teste consiste em agrupar testes semelhantes em uma mesma classe de teste e promover o reuso do código comum através de um método especial e compartilhado entre os testes da classe. Na implementação manual do código de teste essa tarefa é realizada pelo desenvolvedor que, a partir de uma análise preliminar, determina os agrupamentos e promove o reuso do código. Para isso, é necessário que o desenvolvedor realize as seguintes etapas: (1) identifique testes com código comum; (2) agrupe os testes com código em comum em **classes de teste**; e (3) extraia o código comum para um método compartilhado entre os testes da classe. Tipicamente esse processo é realizado de forma incremental, de modo que as etapas mencionadas sejam realizadas a cada novo teste adicionado ao sistema. Porém, também pode ocorrer de forma não incremental, onde esse mesmo processo é aplicado somente após a

implementação de um conjunto de testes. Essa forma de fatoração de testes será explicada mais a frente, na Seção 2.1.1.2.

Identificar itens similares é um problema comum em diversas áreas da computação. Diferentes métricas de similaridade existem para tratar esse problema. Em teste de *software*, métricas de similaridade de código têm sido aplicadas para medir a diversidade entre testes. Tal medida tem sido especialmente útil para os tópicos de redução e priorização de casos de teste onde abordagens baseadas em similaridade vêm sendo propostas (Miranda *et al.*, 2018). Para além da priorização de casos de teste, este trabalho é sustentado pela ideia fundamental de que métricas de similaridade podem ser aplicadas para identificar testes com código em comum. Por exemplo, considerando o teste apresentado na Figura 1 seria possível utilizar métricas de similaridade que permitam identificar que as linhas 4 a 6 também aparecem no teste apresentado na Figura 2. A utilização de métricas de similaridade de código permite que o processo de identificação de testes com código em comum seja, então, automatizado, reduzindo, assim, o esforço de desenvolvimento.

Após a identificação de testes com código em comum, o desenvolvedor ainda precisa realizar o agrupamento dos testes através de classes. Algoritmos de agrupamento ou *clustering* são utilizados para organizar conjuntos de elementos de modo que elementos mais similares de acordo com algum critério definido sejam colocados no mesmo grupo (Jain, Murty e Flynn, 1999). Por exemplo, considerando que o teste apresentado na Figura 1 tenha uma alta similaridade em relação ao teste apresentado na Figura 2, então o algoritmo de agrupamento poderia indicar que ambos os testes fossem colocados na mesma classe com o objetivo de promover o reuso do código de teste. Assim, é possível reduzir ainda mais o esforço de desenvolvimento através da utilização de algoritmos de agrupamento que, considerando os testes semelhantes, identificados a partir das métricas de similaridade, possam criar os melhores grupos desses testes.

Metodologias ágeis de desenvolvimento, como XP, defendem a refatoração constante tanto do código de teste quanto do código da aplicação. De acordo com Opdyke (1992), refatoração consiste no processo de reestruturação de um programa sem que seu comportamento seja alterado, sendo o reuso de código um dos objetivos da refatoração. Diante dessa perspectiva, **este trabalho propõe a refatoração automática de classes de teste usando métricas de similaridade para**

identificação de testes similares e algoritmos de agrupamento para a criação de grupos de testes similares entre si.

1.1. PROBLEMA DE PESQUISA

No contexto da implementação manual de código de teste, técnicas de reuso de código são utilizadas durante a criação e manutenção dos testes para melhorar a manutenibilidade (Fowler, 1999; Meszaros 2007). A aplicação manual de técnicas de reuso de código é suscetível a erros de refatoração e demanda um esforço de desenvolvimento que, por vezes, supera o ganho obtido com a melhoria na manutenibilidade (Landhäußer e Tichy, 2012; Meszaros 2007), principalmente quando o número de testes aumenta (Berner, Weber e Keller, 2005). O problema está em aplicar técnicas de reuso de código de teste reduzindo o esforço de desenvolvimento sem que erros ou inconsistências de refatoração possam ocorrer durante o processo. Diferentes abordagens para esse problema já foram propostas (Van Deursen *et al.*, 2001; Christensen *et al.*, 2006; Longo *et al.*, 2015; Silva e Vilain, 2017). Entretanto, apesar de se mostrarem eficientes para reduzir a duplicação de código de teste, a refatoração através dessas abordagens ainda depende de intervenção manual. Mesmo sendo possível reduzir a duplicação de código de teste ainda não há garantia de redução no esforço total de desenvolvimento dos testes e nem garantia de que não irão ocorrer erros ou inconsistências durante o processo.

1.2. HIPÓTESE DE PESQUISA

Considerando o problema apresentado, a hipótese de pesquisa deste trabalho é enunciada da seguinte forma: através de um método de refatoração automatizado de classes teste que (1) utiliza métricas de similaridade para identificação de testes com código em comum e (2) algoritmos de agrupamento para a criação de grupos de testes similares entre si, é possível reduzir a duplicação de código de teste e, ao mesmo tempo, reduzir o esforço total de desenvolvimento dos testes sem que o processo esteja suscetível a erros ou inconsistências.

1.3. OBJETIVOS

O objetivo geral deste trabalho consiste em reduzir a duplicação de código de teste e reduzir o esforço total de desenvolvimento dos testes sem alterar o resultado esperado dos testes. Para alcançar o objetivo geral, os seguintes objetivos específicos deverão ser alcançados: (1) identificar métricas de similaridade adequadas para identificar testes com código em comum; (2) identificar algoritmos de agrupamento adequados para criar grupos de testes com código em comum entre si; (3) definir um modelo para refatoração automática de classes de teste a partir da utilização das métricas de similaridade e algoritmos de agrupamento; e (4) definir um *framework* para o modelo com base na seleção das métricas de similaridade e algoritmos de agrupamento mais adequados.

1.4. ORGANIZAÇÃO DO TRABALHO

Os demais capítulos deste trabalho são organizados da seguinte forma: no Capítulo 2 é apresentada a fundamentação teórica; no Capítulo 3 é apresentado o estado da arte e os trabalhos relacionados; no Capítulo 4 é apresentada a proposta deste trabalho; e no Capítulo 5 é apresentada a conclusão e os próximos passos para elaboração deste trabalho.

2. FUNDAMENTAÇÃO TEÓRICA

Três tópicos se destacam em nível de importância para este trabalho, nomeadamente teste de *software*, métricas de similaridade e algoritmos de agrupamento. Neste capítulo serão abordados os principais conceitos envolvendo a implementação manual de testes automatizados e as diferentes famílias de métricas de similaridade.

2.1. TESTE DE SOFTWARE

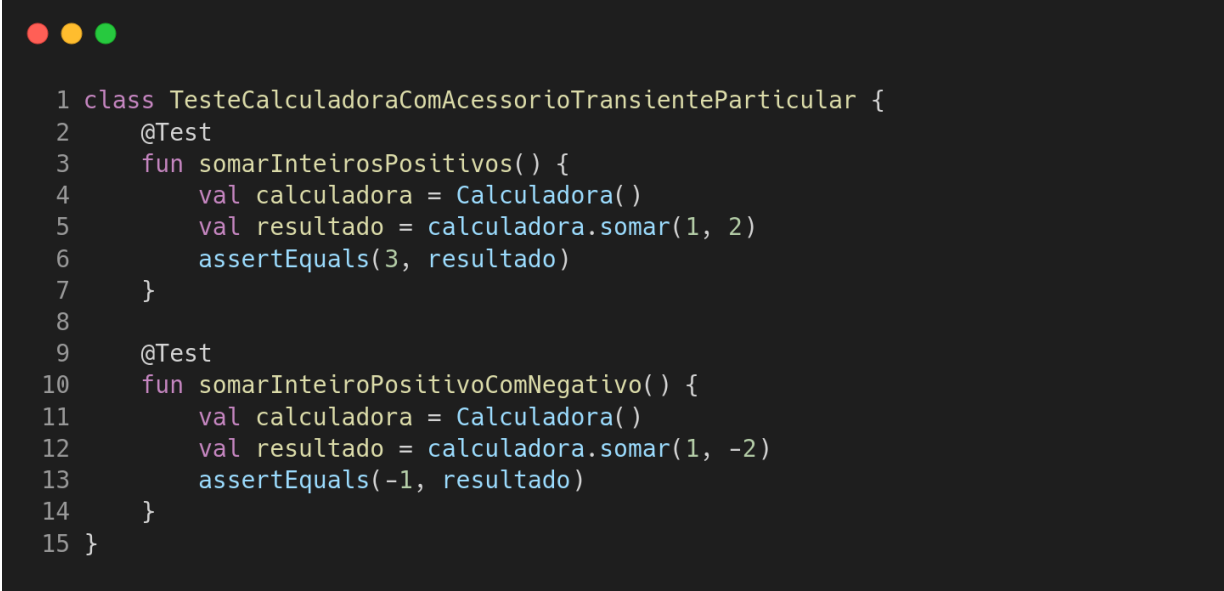
Um teste é um procedimento, manual ou automático, que pode ser usado para verificar se um dado SUT funciona de acordo com o comportamento esperado. O SUT representa a parte do sistema que está sendo testada. Cada teste deve realizar, inicialmente, uma série de passos para colocar o SUT no estado desejado para que o teste possa ser executado. Os elementos necessários para que o SUT seja colocado no estado desejado recebem o nome de *test fixtures*, ou simplesmente *fixtures*, ou acessórios de teste em português. A parte da lógica do teste onde os acessórios de teste são criados é chamada de *fixture setup*, ou configuração de acessórios. Após executar a configuração de acessórios, o teste exercita o SUT através da execução da operação que está sendo testada e, por fim, realiza uma série de verificações para garantir que o SUT funciona de acordo com o comportamento esperado.

Os exemplos apresentados nesse trabalho são baseados no *framework* de teste JUnit, disponível para as linguagens Java e Kotlin. Os exemplos utilizarão a linguagem Kotlin. A Figura 3 apresenta um exemplo contendo uma classe de teste e dois testes. A anotação `@Test` nas linhas 2 e 9 indica ao *framework* que os métodos anotados correspondem a testes. Essa abordagem segue a definição utilizada por Freeman e Pryce (2009), onde, tipicamente, cada teste é separado em um método de teste. Além disso, cada teste pode ser dividido conceitualmente em quatro fases (Meszaros, 2007): configuração (*setup*), exercício, verificação e finalização (*tear down*).

Considerando o primeiro teste da Figura 3, a linha 4 corresponde à etapa de configuração. Nessa etapa, o SUT deve ser colocado no estado desejado para o teste. É na configuração que os acessórios necessários para o teste serão definidos.

Nesse caso, a instância da classe `Calculadora` é um acessório necessário para o teste.

Figura 3. Teste calculadora com dois testes.

A screenshot of a code editor with a dark background and light-colored text. At the top left, there are three colored circles (red, yellow, green) representing window controls. The code is written in Kotlin and defines a class `TesteCalculadoraComAcessorioTransienteParticular` with two test methods. The first method, `somarInteirosPositivos()`, creates a `Calculadora` instance, calls `somar(1, 2)`, and asserts the result is 3. The second method, `somarInteiroPositivoComNegativo()`, creates a `Calculadora` instance, calls `somar(1, -2)`, and asserts the result is -1. The code is numbered from 1 to 15.

```
1 class TesteCalculadoraComAcessorioTransienteParticular {
2     @Test
3     fun somarInteirosPositivos() {
4         val calculadora = Calculadora()
5         val resultado = calculadora.somar(1, 2)
6         assertEquals(3, resultado)
7     }
8
9     @Test
10    fun somarInteiroPositivoComNegativo() {
11        val calculadora = Calculadora()
12        val resultado = calculadora.somar(1, -2)
13        assertEquals(-1, resultado)
14    }
15 }
```

A etapa seguinte em um teste consiste em exercitar o SUT. É nessa etapa que é executado o que efetivamente será testado. Na Figura 3, a linha 5 corresponde à etapa de exercício do primeiro teste. Existem casos, entretanto, em que a separação entre configuração e exercício não é tão bem definida. Um exemplo disso pode ser observado no teste apresentado na Figura 1, onde não existe uma separação clara entre configuração e exercício. Nesses casos as etapas configuração e exercício podem, também, ser vistas como uma única etapa, não havendo, assim, divisão entre elas.

Na etapa de verificação o comportamento esperado do sistema é verificado a partir das saídas produzidas pelo SUT ou através de mecanismos que permitam obter o estado interno do SUT. No primeiro teste da Figura 3 a verificação ocorre na linha 6.

A última etapa não aparece explicitamente nos testes da Figura 3, mas nela os acessórios definidos na etapa de configuração são destruídos para evitar que futuras execuções do teste ou que execuções futuras de outros testes venham a ser afetadas. Nos testes apresentados, a etapa de finalização é executada implicitamente pela própria máquina virtual Java (*Java Virtual Machine* – JVM). Ao

término da execução do método de teste, a JVM se encarrega de remover o acessório `calculadora` da memória.

Uma prática comumente utilizada é agrupar diversos testes em uma mesma classe. O critério de agrupamento dos testes varia conforme o projeto. De acordo com Meszaros (2007), três estratégias de agrupamento se destacam: classe de teste por funcionalidade da aplicação, classe de teste por classe da aplicação e classe de teste por acessório. Nessa última abordagem testes que possuam acessórios semelhantes são agrupados na mesma classe.

2.1.1. Categorias e estratégias de configuração de acessórios

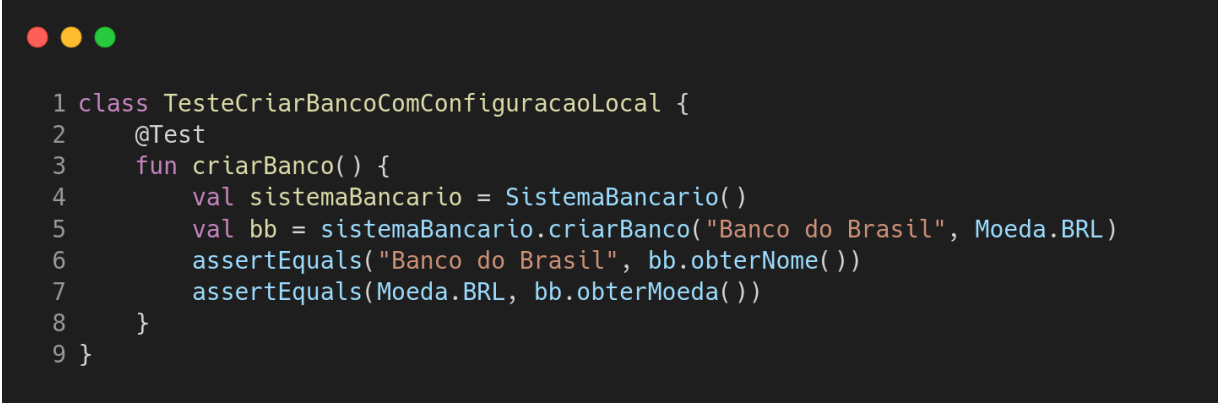
A configuração de acessórios compreende a lógica do teste onde os acessórios são criados. Cada teste possui uma configuração de acessórios, que consiste na criação dos acessórios que serão utilizados no teste. A configuração de acessórios de um teste pode ser realizada através de diferentes estratégias, podendo ser uma ou mais estratégias ao mesmo tempo. Um dos problemas que afetam a manutenibilidade do código de teste é a duplicação do código de configuração de acessórios. Nem sempre é simples e/ou possível evitar esse tipo de problema, porém existem diferentes estratégias de configuração de acessórios que podem ser utilizadas para reduzir a duplicação de código.

Mesaros (2007) divide as estratégias de configuração de acessórios em duas categorias: *fresh fixture setup*, ou configuração de acessórios novos e *shared fixture construction*, ou configuração de acessórios compartilhados. Em ambas as categorias existem estratégias que possibilitam o reuso de código. Entretanto, apenas na configuração de acessórios compartilhados é possível o reuso de execução. A categoria configuração de acessórios novos é composta por três estratégias, sendo elas: *inline setup*, ou configuração local, *implicit setup*, ou configuração implícita, e *delegated setup*, ou configuração delegada. Nessa categoria há apenas a possibilidade de reuso de código, mas não de reuso de execução. Já as estratégias da configuração de acessórios compartilhados realizam a criação de acessórios que podem ser criados uma única vez e reutilizados entre várias execuções de teste distintas.

2.1.1.1. Estratégia configuração local

Na estratégia configuração local os acessórios são criados dentro do próprio método de teste. Em geral, essa estratégia é utilizada em testes que necessitam de acessórios muito específicos ou no desenvolvimento dos primeiros testes, onde ainda não existe a necessidade de reuso de acessórios. A Figura 4 mostra um exemplo de teste em que a estratégia configuração local é utilizada. As linhas 4 e 5 contêm a configuração de acessórios do teste. Destaca-se que na estratégia configuração local a configuração de acessórios é realizada diretamente no método de teste.

Figura 4. Teste com a estratégia configuração local.



```
1 class TesteCriarBancoComConfiguracaoLocal {
2     @Test
3     fun criarBanco() {
4         val sistemaBancario = SistemaBancario()
5         val bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
6         assertEquals("Banco do Brasil", bb.obterNome())
7         assertEquals(Moeda.BRL, bb.obterMoeda())
8     }
9 }
```

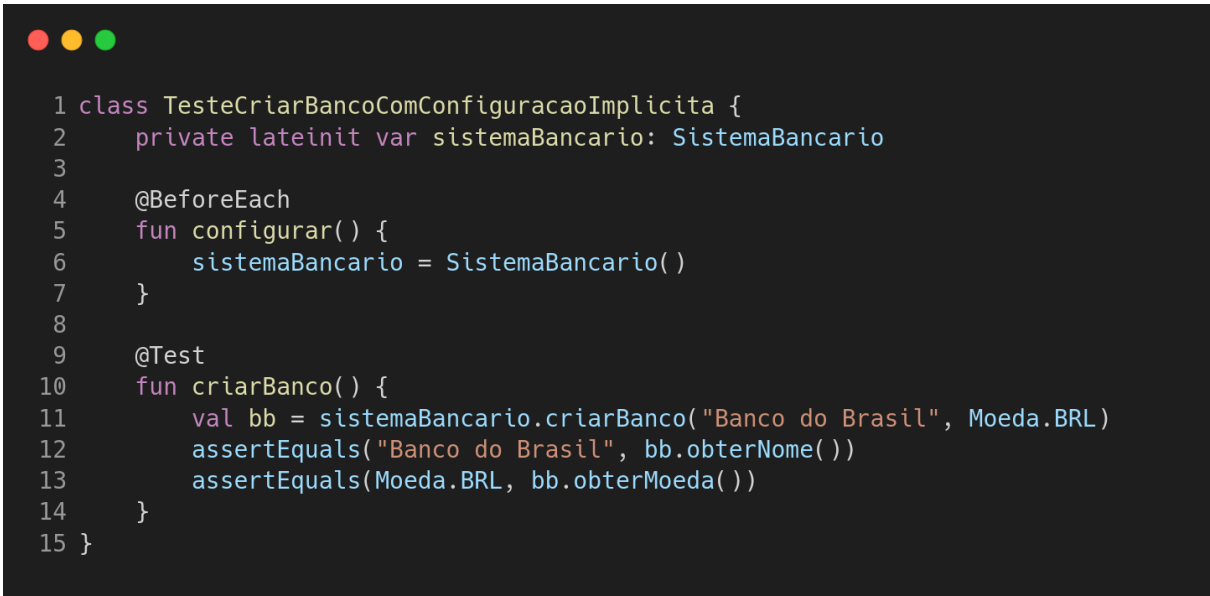
Como os acessórios são criados diretamente no método de teste, torna-se fácil compreender a relação de causa e efeito entre os acessórios e as saídas do SUT. Entretanto, a longo prazo, essa estratégia tende a aumentar a duplicação de código, uma vez que diversos testes podem utilizar acessórios idênticos ou muito similares. Além disso, essa estratégia pode dificultar a compreensão do teste nos casos em que os acessórios forem muito complexos, uma vez que a quantidade de código aumenta e se torna mais difícil compreender o papel do acessório no teste.

2.1.1.2. Estratégia configuração implícita

Na estratégia configuração implícita os acessórios são criados através de um método especial, comumente denominado de *setup method*, ou método de configuração, pertencente à classe de teste e executado antes de cada método de teste. A Figura 5 apresenta um exemplo da estratégia configuração implícita. Nesse

exemplo, o método `configurar` da linha 5 representa o método de configuração. O atributo `bb` declarado na linha 2 é utilizado para permitir que o acessório seja manuseável pelos testes da classe. Quando um teste da classe é executado, o *framework* fica responsável por descobrir e executar o método de configuração. No JUnit a anotação `@BeforeEach` é utilizada para indicar o método de configuração.

Figura 5. Teste com a estratégia configuração implícita.



```

1 class TesteCriarBancoComConfiguracaoImplicita {
2     private lateinit var sistemaBancario: SistemaBancario
3
4     @BeforeEach
5     fun configurar() {
6         sistemaBancario = SistemaBancario()
7     }
8
9     @Test
10    fun criarBanco() {
11        val bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
12        assertEquals("Banco do Brasil", bb.obterNome())
13        assertEquals(Moeda.BRL, bb.obterMoeda())
14    }
15 }

```

A vantagem dessa estratégia é promover o reuso de acessórios entre testes de uma classe. Porém, para utilizar essa estratégia, os testes que utilizam os mesmos acessórios devem ser agrupados na mesma classe. Isso pode trazer uma complicação a longo prazo, pois se torna cada vez mais difícil agrupar testes que necessitem exatamente dos mesmos acessórios. Com facilidade alguns testes irão necessitar de acessórios específicos somente para eles. Além disso, nem sempre é possível evitar a duplicação de código através dessa estratégia, uma vez que um determinado teste pode ter acessórios similares a dois outros testes que, por sua vez, não possuam acessórios similares entre si.

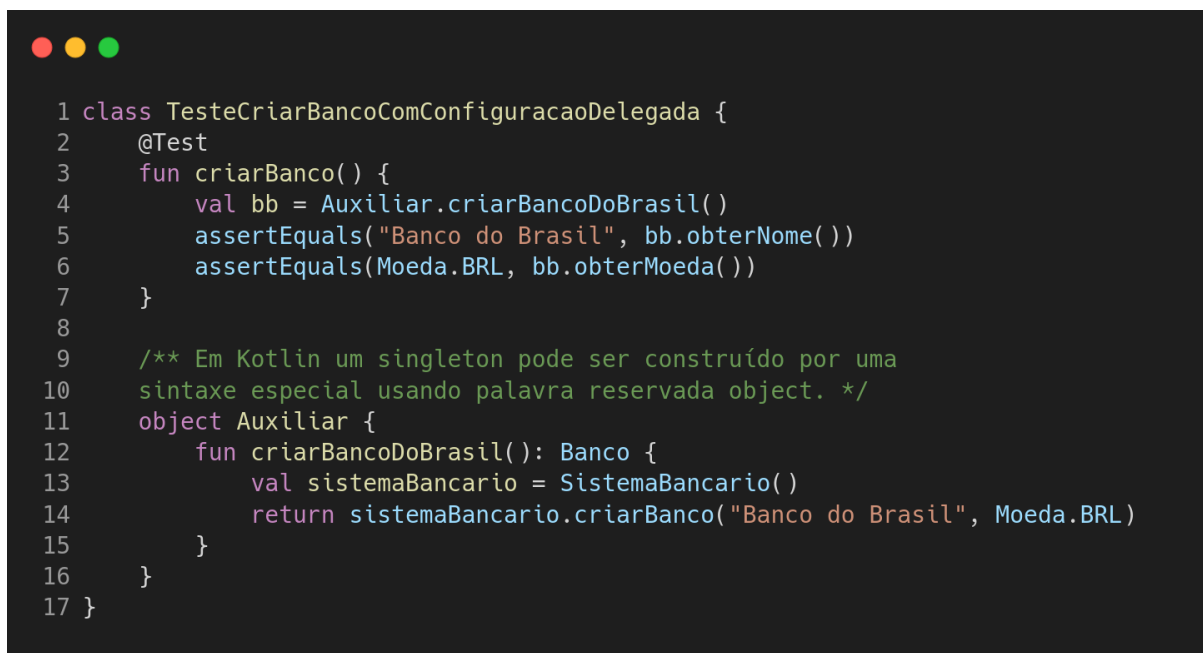
Greiler *et al.* (2013) apresenta *test smells*, ou fedores de teste, que podem dificultar a manutenibilidade do código de teste quando essa estratégia é utilizada. Um dos principais problemas apontados é o *general fixture smell*, ou fedor de acessórios generalizados, que ocorre quando existem acessórios dentro do método de configuração que são necessários apenas para alguns testes da classe. Isso

pode prejudicar a legibilidade dos demais testes, uma vez que existirá mais acessórios do que apenas os necessários.

2.1.1.3. Estratégia de configuração delegada

A estratégia configuração delegada utiliza métodos auxiliares responsáveis pela criação dos acessórios. Em geral, esses métodos são disponibilizados através de classes auxiliares, separadas das classes de teste. A Figura 6 apresenta um exemplo da estratégia configuração delegada. O acessório necessário para o teste é criado através do método `criarBancoDoBrasil`, definido na linha 2 da classe `Auxiliar`. Esse método auxiliar é chamado na linha 4 da classe de teste.

Figura 6. Teste com a estratégia configuração delegada.



```
1 class TesteCriarBancoComConfiguracaoDelegada {
2     @Test
3     fun criarBanco() {
4         val bb = Auxiliar.criarBancoDoBrasil()
5         assertEquals("Banco do Brasil", bb.obterNome())
6         assertEquals(Moeda.BRL, bb.obterMoeda())
7     }
8
9     /** Em Kotlin um singleton pode ser construído por uma
10     sintaxe especial usando palavra reservada object. */
11     object Auxiliar {
12         fun criarBancoDoBrasil(): Banco {
13             val sistemaBancario = SistemaBancario()
14             return sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
15         }
16     }
17 }
```

A principal vantagem dessa estratégia é permitir que o mesmo código de teste possa ser utilizado por testes diferentes de classes diferentes. A desvantagem dessa estratégia é que, em alguns casos, a sua utilização pode dificultar a compreensão da relação de causa e efeito entre os acessórios e as saídas do SUT, uma vez que o método auxiliar é, comumente, colocado em uma classe separada do teste.

2.1.1.4. Categoria de estratégias de configuração de acessórios compartilhados

As estratégias da categoria configuração de acessórios compartilhados realizam a criação de acessórios coletivos. Dessa forma, um acessório pode ser criado uma única vez e reutilizado entre várias execuções de teste distintas. Os *frameworks* de teste atuais não oferecem formas nativas e nem padronizadas para realizar esse tipo de configuração de acessórios. Por isso, é necessário que o desenvolvedor do teste assuma esse papel e passe a gerenciar parte do fluxo de execução dos testes.

2.2. MÉTRICAS DE SIMILARIDADE

Métricas de similaridade possuem aplicações em diversas áreas da computação, como reconhecimento facial em imagens (Cao, Ying e Li, 2013), busca textual (Sahami e Heilman, 2006), recuperação de informação estruturada (Dorneles *et al.*, 2004), entre outros. Métricas de similaridade determinam o grau de similaridade entre dois elementos, onde grau de similaridade é um valor entre 0 e 1 que indica o quanto dois elementos são parecidos entre si de acordo com o critério definido pela métrica de similaridade. Cada métrica de similaridade possui um mecanismo para gerar o grau de similaridade de modo que cada métrica apresenta uma distribuição específica sobre o domínio de valores. Neste trabalho, pretende-se avaliar a similaridade entre testes através do código de teste. Por isso, métricas de similaridade de código serão abordadas a seguir.

2.2.1. Métricas de similaridade de código

Avaliar a similaridade de código é uma atividade que possui propósitos variados, como identificação de fragmentos duplicados de código (Roy e Cordy, 2008), detecção de plágio e verificação de violação de direitos de cópia de *software* (Ottenstein, 1976), recomendação de código (Holmes e Murphy, 2005) e outros. Ragkhitwetsagul, Krinke e Clark (2018) classificam cinco diferentes abordagens de métricas de similaridade de código, sendo elas: baseadas em medidas do código, baseadas em texto, baseadas em *tokens*, baseadas em árvore e baseadas em

grafo. Nas seções a seguir essas cinco abordagens, bem como exemplos de ferramentas, são apresentadas.

2.2.1.1. Métricas baseadas em medidas do código

Medidas do código como, por exemplo, a quantidade de linhas e a quantidade de variáveis entre duas funções diferentes, podem ser usadas como indicativos de similaridade de código. As abordagens baseadas em medidas do código foram as primeiras relatadas na literatura e foram aplicadas na detecção de similaridade de *software*. Ottenstein (1976) utiliza métricas de similaridade para identificar potenciais equivalências entre programas de alunos de uma disciplina de programação. O problema de identificação de plágios entre trabalhos de disciplinas de programação também é abordado por Donaldson, Lancaster e Sposato (1981). Além dessas, outras abordagens baseadas em medidas de código são encontradas na literatura (Grier, 1981; Berghel e Sallach, 1984; Faidhi e Robinson, 1987).

2.2.1.2. Métricas baseadas em texto

Métricas de similaridade de código baseadas em texto avaliam a similaridade entre dois programas através da análise de sequências de caracteres do código fonte. Roy e Cordy (2008) propõem a identificação de clones de fragmentos de código através de uma técnica que analisa diretamente o código fonte. Diferente de abordagens que buscam identificar plágio previamente apresentadas, a proposta busca encontrar fragmentos de código intencionalmente copiados com o propósito de reuso. A proposta não tem como objetivo capturar a renomeação de identificadores, uma vez que parte do pressuposto que identificadores presentes em fragmentos de código clonados intencionalmente para evitar retrabalho, normalmente, não são renomeados a menos que isso seja necessário. Dessa forma, a proposta pretende identificar apenas clones que foram modificados de forma intencional para que o fragmento possa se adaptar ao contexto em que foi inserido.

2.2.1.3. Métricas baseadas em tokens

Assim como as abordagens baseadas em texto, as abordagens baseadas em *tokens* também avaliam a similaridade entre dois programas através da análise textual do código fonte. Abordagens baseadas em *tokens*, no entanto, aumentam o nível de abstração convertendo o código fonte em sequências de *tokens*. A utilização de *tokens* visa normalizar diferenças textuais através da definição de tipos abstratos de constructos textuais. Por exemplo, todos os identificadores ou valores literais presentes no código podem ser representados por um mesmo *token*.

Kamiya, Kusumoto e Inoue (2002) utilizam a abordagem baseada em *token* na ferramenta CCFinder. A ferramenta é capaz de identificar clones em diferentes linguagens, incluindo C, C++, Java e COBOL.

2.2.1.4. Métricas baseadas em árvores

Diferente das categorias vistas anteriormente, esta categoria de técnicas tem o foco voltado para identificar a similaridade entre dois programas com base na semelhança estrutural entre eles. Nesse sentido, diferenças léxicas são menos relevantes na análise de similaridade. Para avaliar a similaridade entre dois programas são construídas árvores abstratas de sintaxe (*Abstract Syntax Tree* – AST). As ASTs permitem identificar a similaridade entre dois trechos mesmo com uma alta quantidade de modificações entre eles. A complexidade computacional, entretanto, é mais alta quando comparada com as categorias de técnicas vistas anteriormente. Baxter *et al.* (1998) utilizam ASTs para identificar clones, ou clones que sofreram modificações, no contexto de programas escritos na linguagem C.

2.2.1.5. Métricas baseadas em grafo

Árvores podem ser vistas como uma especialização de grafos. Nesse sentido métricas de similaridade baseadas em árvores se assemelham às métricas baseadas em grafos, uma vez que ambas permitem capturar aspectos estruturais do código fonte. As métricas baseadas em grafo, no entanto, possuem como característica adicional a capacidade de identificar similaridade de aspectos semânticos. O maior problema nas métricas baseadas em grafo está na alta

complexidade computacional, em geral, ainda maior do que nas métricas baseadas em árvores. Krinke (2001) utiliza grafos para representar a estrutura e o fluxo de dados de programas e, a partir disso, identificar duplicação de código. O objetivo é maximizar a identificação de trechos de código duplicados ainda que exista diferença entre os trechos de código, minimizando ao máximo a ocorrência de falsos positivos. Isto é, evitar a identificação de dois trechos de código como duplicados quando na verdade não são. Komondoor e Horwitz (2001) também representam o código fonte através de PGDs para identificar duplicação de código.

3. ESTADO DA ARTE

Para levantamento do estado da arte e identificação de trabalhos correlatos foi realizada uma revisão exploratória da literatura, seguida de um estudo sistemático da literatura. A primeira teve como principal objetivo a construção de uma base de conhecimento mais aprofundada sobre os temas relacionados com este trabalho, como teste de *software*, reuso de código, métricas de similaridade e algoritmos de agrupamento, bem como verificar a interseção entre essas áreas como, por exemplo, trabalhos que abordam teste de *software* e métricas de similaridade. Já no estudo sistemático o foco foi identificar trabalhos que possuam um objetivo similar ao objetivo deste trabalho. Isto é, trabalhos voltados para a refatoração do código de teste. O objetivo principal da segunda etapa foi verificar a originalidade desta proposta, isto é, se este trabalho tem potencial para contribuir com o estado da arte das áreas envolvidas, principalmente no campo de teste de *software*.

3.1. REVISÃO EXPLORATÓRIA

O tema do trabalho foi explorado através da avaliação de trabalhos considerados empiricamente com maior relevância para esta pesquisa. Para a seleção dos trabalhos mais relevantes foram utilizadas as seguintes bases de pesquisa: ACM Digital Library¹, Springer², IEEE Xplore³ e ScienceDirect⁴. Além disso, a partir dos trabalhos selecionados, as referências consideradas mais relevantes para esta pesquisa também foram consultadas. Inicialmente este trabalho focou na busca por técnicas de reuso de código de teste, refatoração de código de teste e métricas de similaridade. Conforme o escopo deste trabalho foi se tornando mais específico, novos tópicos, como métricas de similaridade de código e a aplicação dessas métricas na área de teste de *software*, também foram pesquisados.

¹ <https://dl.acm.org>

² <https://springer.com>

³ <https://ieeexplore.ieee.org>

⁴ <https://sciencedirect.com>

Convém destacar que além da revisão exploratória também foram utilizadas referências previamente conhecidas para a elaboração da fundamentação teórica presente neste trabalho.

3.1.1. Teste de software e métricas de similaridade

Um dos passos iniciais para a avaliação do estado da arte envolvendo o tema deste trabalho consistiu em investigar a aplicação de métricas de similaridade no contexto de teste de *software*. Através da pesquisa exploratória, dois tópicos de teste de *software* se destacam na utilização de métricas de similaridade, nomeadamente, redução de suíte de teste (*test suite reduction*) e priorização de caso de teste (*test case prioritization*).

Redução de suíte de teste (Harrold, Gupta e Soffa, 1993) tem como objetivo acelerar a execução de testes de regressão através da seleção, de acordo com algum critério de seleção estabelecido, de apenas um subconjunto dos testes de uma dada suíte. O objetivo, portanto, é reduzir o conjunto de **casos de teste** a serem executados, reduzindo, assim, o tempo total de execução dos testes. Priorização de caso de teste (Wong *et al.*, 1997), por sua vez, ao invés de remover testes da suíte, tem como objetivo ordenar o conjunto de **casos** de teste de acordo com algum critério de ordenação estabelecido.

3.1.2. Reuso de código de teste

Na sequência serão apresentados trabalhos que propõem técnicas de reuso de código de teste.

3.1.2.1. DbUnit

Christensen *et al.* (2006) apresentam o DbUnit, uma extensão do *framework* de teste JUnit que possibilita reutilizar acessórios persistentes e coletivos em testes que envolvem a utilização de tabelas do banco de dados. Na extensão proposta são declaradas dependências entre classes de teste. Essas dependências devem refletir a mesma dependência existente entre as tabelas do banco de dados. Assim, os testes das dependências de uma determinada classe são executados de forma

recursiva antes da execução dos testes da própria classe. O trabalho considera que deverá ser definida uma classe de teste para cada tabela do banco de dados que será testada. Na abordagem adotada é necessário que também sejam definidos testes para as tabelas que forem chaves estrangeiras de tabelas testadas. Dessa forma, antes de executar um teste onde é realizada a inserção de um dado, primeiro será executado um teste onde o dado que corresponde à chave estrangeira é inserido. A abordagem define que para cada teste que realiza a inserção de um registro deverá ser definido um teste para remover o registro inserido. Essa restrição é adotada para que o banco de dados seja sempre mantido em um estado consistente.

3.1.2.2. *Picon*

O trabalho proposto por Longo *et al.* (2015) utiliza uma linguagem de notação de objetos baseada em JSON para definir os acessórios. Os acessórios são definidos através de arquivos com a extensão `picon` que ficam em um local separado dos testes. O objetivo do trabalho é criar um repositório centralizado de acessórios que poderão ser utilizados por qualquer teste. Cada acessório deve ser identificado através de um qualificador. Os acessórios de teste se tornam manuseáveis para os testes através da declaração de um atributo na classe de teste. O atributo deve ser nomeado de acordo com o qualificador do acessório desejado. Por exemplo, se um acessório qualificado como `programador:joao`, tal acessório poderá ser manuseado pelo código de teste através da declaração de um atributo de nome `programadorJoao`. Uma das limitações da abordagem é que, como a injeção deles é realizada dinamicamente, eventuais inconsistências entre os qualificadores dos acessórios e os nomes dos atributos somente serão percebidas somente durante a execução dos testes. Outra limitação é que a configuração dos acessórios deve ser descrita através de uma linguagem específica, diferente da linguagem na qual o teste é escrito.

3.1.2.3. *Estória*

A abordagem proposta por Silva e Vilain (2007) busca promover o reuso de código de teste entre classes de teste diferentes. A proposta define o conceito de

dependência entre classes de teste para que testes de uma determinada classe possam reutilizar os acessórios definidos em outra classe de teste. Com isso, a proposta permite a criação de hierarquias de classes de teste, de modo que uma classe na base de uma determinada hierarquia tenha acesso aos acessórios de todas as classes que estão mais ao topo da hierarquia. A relação de dependência entre duas classes de teste implica que a classe dependente terá acesso à configuração de acessório da outra classe.

Além do reuso do código de teste, a proposta também possibilita que seja realizado o reuso de execução. O reuso de execução pode ocorrer para os testes que não alteram o estado interno do SUT. Nesses casos, é possível promover o reuso de execução dos testes que pertencem a um mesmo ramo da hierarquia de classes de teste sem que seja violado o princípio de independência dos testes (Meszaros *et al.*, 2003). Para isso, a abordagem propõe a utilização de uma anotação (`@Safe`) que indique que um teste pode ter sua execução reutilizada com segurança. Nesse caso, o *framework* de teste irá reusar, sempre que possível, a execução dos testes marcados como seguros.

3.1.3. Comparação entre os trabalhos

Esta seção apresenta uma comparação entre as técnicas de reuso de código de teste e a abordagem proposta neste trabalho. Além das abordagens apresentadas na Seção 3.1.2, consideraremos também as estratégias de configuração implícita (Seção 2.1.1.2) e delegada (Seção 2.1.1.3). Em especial, busca-se analisar quatro fatores considerados importantes para este trabalho, sendo eles: escopo de uso, forma de identificação de duplicação, processo de refatoração e esforço de desenvolvimento. Cada um desses fatores é explicado detalhadamente nas seções a seguir.

3.1.3.1. Escopo

Cada técnica de reuso de código de teste é limitada a um escopo específico. Por exemplo, algumas técnicas podem ser aplicadas apenas a testes que envolvem a utilização de acessórios que são persistidos no banco de dados, enquanto outras técnicas podem ser aplicadas apenas ao código que inicializa objetos, e assim por

diante. A compreensão do escopo é importante para este trabalho, pois determina a abrangência e a granularidade de cada técnica. Quanto mais abrangente for uma técnica, maiores serão as oportunidades de reuso, e quanto maior for a granularidade, maior será a quantidade de código que poderá ser reusado considerando a mesma oportunidade de reuso. A Tabela 1 apresenta a relação o escopo de cada uma das técnicas.

Tabela 1. Escopo de uso de técnicas de reuso.

Técnica	Escopo
DbUnit (Christensen <i>et al.</i> , 2006)	Código para inserir entidades do banco de dados.
Picon (Longo <i>et al.</i> , 2015)	Código para criação de POJOs.
Configuração implícita (Meszaros, 2007)	Sequências iniciais do código de teste.
Configuração delegada (Meszaros, 2007)	Trechos contíguos do código de teste.
Estória (Silva e Vilain, 2017)	Sequências iniciais do código de teste.
Proposta deste trabalho	Sequências iniciais do código de teste.

3.1.3.2. Forma de identificação de duplicação

A forma de identificação de duplicação é um fator importante, pois impacta diretamente no esforço necessário para identificar possibilidades de reuso. A identificação de duplicação ocorre antes do processo de refatoração e determina qual parte do código deverá ser refatorada. A duplicação pode ser identificada diretamente através de uma análise manual das linhas de código ou pode ser identificada de outras formas como, por exemplo, através da identificação de testes que utilizam as mesmas tabelas do banco de dados. A Tabela 2 apresenta a relação entre as técnicas de reuso de código de teste e a forma de identificação de duplicação.

Tabela 2. Forma de identificação de duplicação de técnicas de reuso.

Técnica	Forma de identificação de duplicação
DbUnit (Christensen <i>et al.</i> , 2006)	Identificação manual de testes que envolvem as mesmas entidades do banco de dados.
Picon (Longo <i>et al.</i> , 2015)	Identificação manual de POJOs que são iguais entre si.
Configuração implícita (Meszaros, 2007)	Identificação manual de sequências iniciais do código de teste que são iguais entre si.
Configuração delegada (Meszaros, 2007)	Identificação manual de trechos contíguos do código de teste que são iguais entre si.
Estória (Silva e Vilain, 2017)	Identificação manual de sequências iniciais do código de teste que são iguais entre si.
Proposta deste trabalho	Identificação automática de sequências iniciais do código de teste que são iguais entre si.

3.1.3.3. Processo de refatoração

A Tabela 3 apresenta um resumo de como ocorre o processo de refatoração em cada uma das técnicas consideradas importantes para este trabalho. Quanto mais simplificado for o processo de refatoração, menor será o esforço necessário para remover um único par de configurações de acessórios duplicados. Com exceção da abordagem proposta por Longo *et al.* (2015), as demais abordagens apresentadas na Tabela 3 possuem um processo semelhante – todas elas envolvem a definição de uma classe e de um método onde o código de teste a ser reusado será colocado.

Tabela 3. Processo de refatoração de técnicas de reuso.

Técnica	Processo de refatoração
DbUnit (Christensen <i>et al.</i> , 2006)	Definição manual dos testes envolvendo uma mesma entidade do banco de dados em uma mesma classe de teste.
Picon (Longo <i>et al.</i> , 2015)	Definição manual do objeto em um arquivo externo e declaração do objeto como atributo das classes de teste que

	o utilizam.
Configuração implícita (Meszaros, 2007)	Definição manual da sequência inicial do código de teste em uma classe e agrupamento dos testes que possuem a mesma sequência inicial do código de teste na mesma classe.
Configuração delegada (Meszaros, 2007)	Definição manual de um método contendo o trecho contíguo do código de teste e declaração de chamada de método nos testes que possuem o mesmo trecho contíguo de código.
Estória (Silva e Vilain, 2017)	Definição manual da sequência inicial do código de teste em uma classe e agrupamento dos testes que possuem a mesma sequência inicial do código de teste na mesma classe.
Proposta deste trabalho	Definição automática da sequência inicial do código de teste em uma classe e agrupamento dos testes que possuem a mesma sequência inicial do código de teste na mesma classe.

3.1.3.4. Esforço de refatoração

O sucesso a longo prazo da utilização de uma determinada técnica de reuso vai ser fortemente influenciado pelo esforço de refatoração. A Tabela 4 apresenta um resumo do esforço de fatoraço das técnicas consideradas relevantes para este trabalho. O esforço de refatoração pode variar conforme o momento projeto, sendo possível que esse esforço aumente ou diminua conforme condições e características específicas do projeto.

Tabela 4. Esforço de refatoração de técnicas de reuso.

Técnica	Esforço de refatoração
DbUnit (Christensen <i>et al.</i> , 2006)	Diminui conforme aumenta a quantidade de testes que utilizam a mesma tabela do banco de dados.
Picon (Longo <i>et al.</i> , 2015)	Diminui conforme aumenta a quantidade de POJOs idênticos que são utilizados pelos testes.

Configuração implícita (Meszaros, 2007)	Diminui conforme diminui a quantidade de grupos de teste que compartilham a mesma configuração inicial de acessórios.
Configuração delegada (Meszaros, 2007)	Diminui conforme aumenta a quantidade de testes que usam o mesmo trecho de configuração de acessórios e conforme o tamanho dessas configurações aumenta.
Estória (Silva e Vilain, 2017)	Diminui conforme diminui a quantidade de grupos de teste que compartilham a mesma configuração inicial de acessórios e conforme aumenta a quantidade de classes de teste dependentes entre si.
Proposta deste trabalho	Por ser automática não existe esforço de desenvolvimento

3.1.4. Discussão

Diferentes fatores são determinantes para a eficiência e eficácia de uma determinada técnica de reuso de código de teste. Uma das principais dificuldades está em garantir que o esforço de refatoração não supere o ganho obtido com a melhoria da manutenibilidade do código. O esforço de refatoração pode aumentar ou diminuir conforme a técnica de reuso utilizada e conforme as características do projeto no momento da refatoração. Isso exige um cuidado constante por parte do desenvolvedor do teste que deve avaliar se determinada técnica de reuso tem sido eficiente ou não para evitar a duplicação de código sem aumentar o esforço total de desenvolvimento dos testes. Além disso, outro problema está no fato de que a identificação de oportunidades de reuso depende diretamente da expertise e experiência do desenvolvedor. Mesmo que algumas técnicas facilitem esta tarefa, em todas elas o sucesso na identificação de duplicações de teste está limitado pela capacidade do desenvolvedor.

A proposta deste trabalho evita esse problema eliminando o fator humano através da automatização do processo de refatoração. A eficiência da proposta no sentido de reduzir a duplicação de código ainda deverá ser avaliada, entretanto, a principal diferença da proposta deste trabalho em relação às outras técnicas de reuso de código de teste está em garantir que o esforço total de desenvolvimento

dos testes não aumente para que seja possível reduzir a duplicação do código de teste.

3.2. ESTUDO SISTEMÁTICO

Além da revisão exploratória da literatura, foi realizado, também, um estudo sistemático visando identificar trabalhos relacionados. A finalidade do estudo sistemático foi identificar se a presente proposta possui originalidade e se tem potencial para trazer contribuições para o estado da arte. Os trabalhos alvos foram aqueles que envolvem a refatoração automática ou semiautomática de **casos de teste**. Consideramos aqui testes escritos através de linguagens de programação, testes baseados em especificação, escritos através de linguagens de domínio específico (*Domain-Specific Language* – DSL), e até mesmo testes escritos em linguagem natural.

Para realizar a identificação dos trabalhos relacionados foram utilizadas as seguintes bases de pesquisa: ACM Digital Library⁵, IEEE Xplore⁶ e Scopus⁷. A expressão de busca utilizada foi: (testing) AND (refactoring OR reuse) AND (automatic OR tool). A expressão foi adaptada para cada base de pesquisa. Para a consulta foram consideradas as seguintes restrições: (1) pesquisar nos títulos, resumos e palavras-chave; (2) apenas trabalhos da área de Ciência da Computação; (3) apenas publicação em revistas e conferências; e (4) apenas trabalhos escritos na língua inglesa. A consulta nas três bases foi realizada **em 08 de junho de 2024**. Foram encontradas **913** publicações, sendo 59 da ACM Digital Library, 66 da IEEE Explore e **808 da Scopus**.

Após a seleção das publicações, como metodologia dividimos a análise em três etapas, conforme apresentado na Figura 7. Na **primeira etapa**, foram lidos os títulos e resumos e respondemos a seguinte pergunta para cada uma das 913 publicações: “A publicação aborda refatoração ou reuso de casos de teste?”. O objetivo desta etapa foi eliminar publicações que não possuísem relação direta com o tema deste trabalho. Esta etapa foi incluída na metodologia pois os termos da expressão de busca podem ser utilizados em contextos distintos. A pergunta foi

⁵ <https://dl.acm.org>

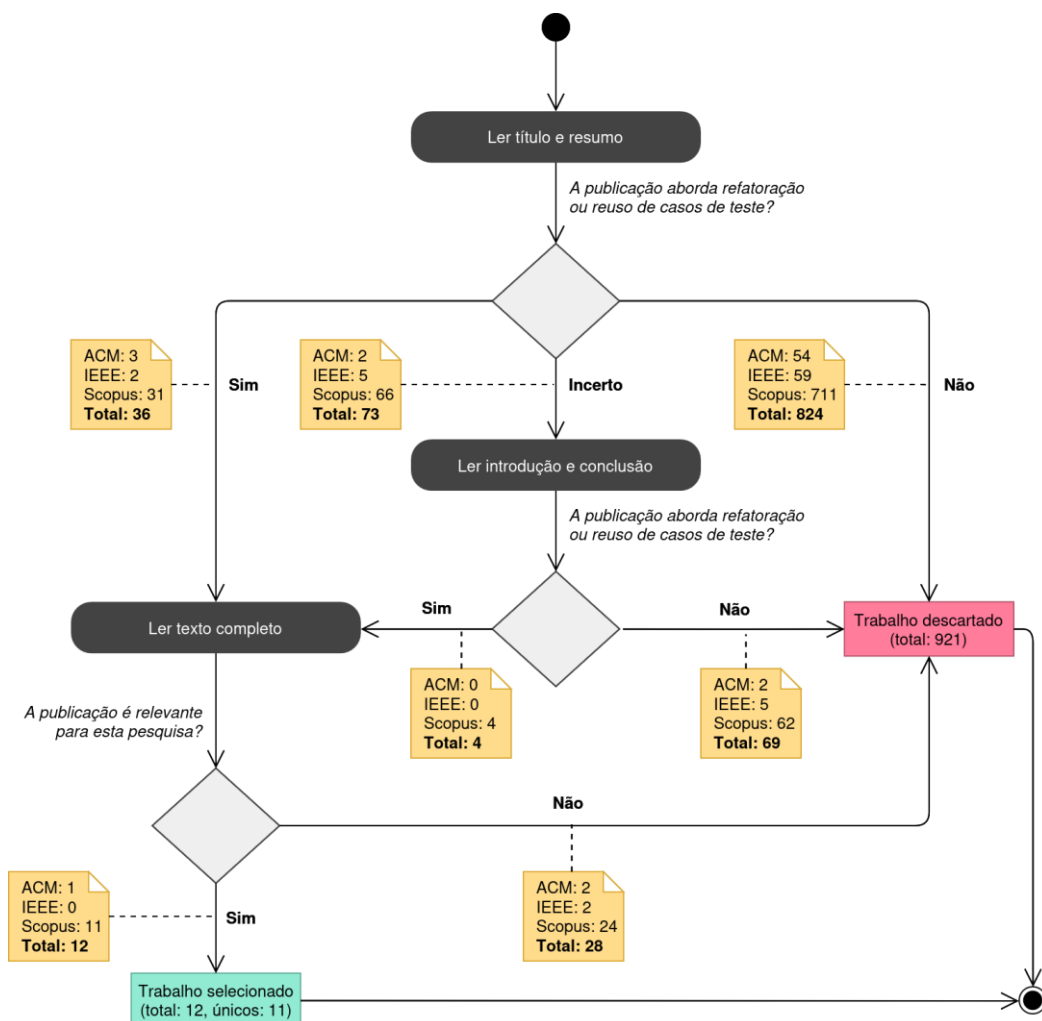
⁶ <https://ieeexplore.ieee.org>

⁷ <https://scopus.com>

respondida com “Sim”, “Não” ou “Incerto”. Caso a pergunta fosse respondida com “Não”, descartávamos a publicação imediatamente. No total, 824 publicações foram descartadas já na primeira etapa. Já a resposta “Sim” (36 no total) levava a publicação diretamente até a última etapa que será explicada mais adiante. A resposta “Incerto” foi usada quando não foi possível afirmar com certeza se a publicação aborda ou não refatoração ou reuso de casos de teste. As 76 publicações com respostas “Incerto” não foram descartadas, porém também não foram incluídas diretamente na última etapa. Essas publicações foram avaliadas na **segunda etapa**, uma espécie de repescagem que consistiu na leitura da introdução e da conclusão das publicações. Assim, a partir da leitura complementar, repetimos na segunda etapa a mesma pergunta feita na primeira etapa, porém, dessa vez, as respostas possíveis foram apenas “Sim” ou “Não”. A maior parte das respostas foram convertidas em “Não” (69 no total) sendo as respectivas publicações descartadas. Apenas 4 publicações tiveram “Sim” como resposta para a pergunta e foram incluídas na terceira e última etapa. Ao todo, 40 trabalhos foram lidos completamente na **terceira etapa**, 36 provenientes diretamente da primeira etapa e 4 provenientes da segunda. Nessa etapa o objetivo foi selecionar apenas publicações relacionadas diretamente com o tema e, ao menos indiretamente, com o objetivo deste trabalho. Para isso, após a leitura completa de cada publicação foi respondida a seguinte pergunta: “*A publicação trata de uma abordagem automática ou semiautomática de refatoração ou reuso de testes de software?*”. Do total, 12 publicações foram consideradas relevantes. Uma das publicações retornadas pela Scopus, também foi retornada pela ACM Digital Library, o que resultou 11 publicações únicas consideradas relevantes. A seguir as 11 publicações serão discutidas.



Figura 7. Diagrama de atividades com as etapas do estudo sistemático.



Três publicações classificadas como relevantes para esta pesquisa são parte do mesmo trabalho proposto por Wang *et al.* (2021, 2022a, 2022b). Os autores abordam os mecanismos de implementação de *mocks*. *Mocks* são implementações falsas (Spadini, 2017) que tem o poder de simular dependências da unidade sendo testada. Herança é um dos mecanismos utilizados para criar *mocks*. Isso porque ao criar uma subclasse da dependência é possível sobrescrever o método que será utilizado no teste e substituir por uma implementação falsa. Entretanto, os autores apontam que herança não é o mecanismo mais adequado para a criação *mocks*, sendo o acoplamento com o código de aplicação um dos principais problemas apontados. Os autores propõem que os *mocks* que utilizam o mecanismo de herança sejam substituídos por *mocks* criados a partir de bibliotecas específicas

para tal, como EasyMock⁸, Mockito⁹ e PowerMock¹⁰. Os autores apresentam uma proposta que permite refatorar o código de teste e substituir de forma automática *mocks* baseados em herança por *mocks* provenientes de bibliotecas. Utilizando projetos de código aberto, a abordagem identificou automaticamente 334 candidatos a refatoração e conseguiu refatorar 276 deles (83%) de forma automática.

Shi, Huang e Wan (2022) propõem um método de reuso orientado a agrupamento para casos de teste escritos em linguagem natural. A abordagem utiliza um algoritmo de agrupamento, denominado *spectral clustering* (Von Luxburg, 2007), para agrupar os casos de teste em pacotes. Para isso, palavras chaves dos dados dos casos de teste são extraídas. Após a extração das palavras chaves, uma matriz de similaridade é gerada e, por fim, o algoritmo de agrupamento é aplicado. O resultado desse processo são os pacotes de casos de teste. O objetivo da proposta é que cada pacote contenha casos de teste com alta similaridade semântica entre si. Vale destacar que a proposta não abrange o código de teste diretamente, mas apenas os casos de teste escritos em linguagem natural. Essa característica traz uma distinção significativa em relação ao nosso trabalho, uma vez que a presente pesquisa tem como foco o agrupamento de testes escritos em linguagens de programação.

Bernard *et al.* (2020) possui uma abordagem similar à apresentada por Shi, Huang e Wan (2022). Nela os casos de teste escritos em linguagem natural são agrupados de acordo com o escopo funcional. Os autores apresentam uma ferramenta, denominada Orbiter, que possibilita diferentes tipos de refatoração dos casos de teste, como: (1) combinar passos similares quando dois passos representam a mesma ação, porém estão escritos de forma diferente; e (2) parametrizar passos, quando dois ou mais passos são iguais entre si, exceto por algum valor que pode ser parametrizado. O agrupamento realizado serve justamente como entrada para essas funcionalidades de refatoração. Porém, assim como ocorre em Shi, Huang e Wan (2022), o trabalho também não aborda a refatoração do código de teste em si, mas apenas dos casos de teste escritos em linguagem natural.

⁸ <https://easymock.org>

⁹ <https://mockito.org>

¹⁰ <https://powermock.github.io>

Santana et al. (2022) apresenta uma abordagem para refatorar dois tipos de fedores de teste, *assertion roulette* e *duplicate assert*. O primeiro ocorre quando um teste possui mais de uma asserção, porém não existe uma mensagem de descrição para cada asserção, tornando mais difícil e trabalhosa a identificação do problema quando o teste falha. Para refatorar esse tipo de fedor de código a ferramenta apresenta os locais onde o fedor ocorre e o usuário deve digitar uma descrição para a asserção, sendo essa uma refatoração semiautomática. Já o segundo ocorre quando o mesmo método do SUT é verificado duas ou mais vezes no mesmo teste. Normalmente, nesse caso, cada chamada do método de teste terá parâmetros diferentes. Para refatorar automaticamente, a ferramenta extrai e divide o teste em múltiplos testes. Entretanto, tal extração irá gerar duplicação de código de teste, uma vez que a ferramenta não promove o reuso de código, seja através da estratégia de configuração implícita ou da estratégia de configuração delegada. Os autores realizaram um experimento no qual participantes refatoraram o código manualmente e utilizando a ferramenta. O tempo médio de refatoração de *assertion roulette* foi de 78,30 segundos na abordagem manual e 49,9 segundos quando utilizada a ferramenta. Já para *duplicate assert*, os respectivos tempos foram de 107,2 e 2,55 segundos.

Xuan et al. (2016) apresenta uma abordagem de refatoração automática do código de teste utilizada para análise dinâmica. Análise dinâmica consiste no processo de analisar um programa enquanto os testes estão em execução. Seu objetivo é variado, podendo incluir a busca por erros e bugs, avaliar a performance, corrigir erros no código de produção de forma automática, entre outros. Xuan et al. (2016) apresentam uma ferramenta de refatoração automática do código de teste, denominada B-Refactoring, que tem como o objetivo melhorar a eficiência de uma ferramenta de análise dinâmica conhecida como Nopol. Através da análise dinâmica de casos de teste, Nopol é capaz de corrigir erros no código de aplicação. Entretanto, os casos de teste precisam exercitar no SUT apenas um caminho de execução. Por exemplo, considere testes para uma função que possui uma condição *if-then-else*. Para que a ferramenta Nopol possa ser aplicada corretamente, é necessário que existam testes separados para o ramo *then* e para o ramo *else*. Caso exista um único teste que exercite as duas condições, a ferramenta Nopol não será capaz de encontrar o erro no código de aplicação e consequentemente não será possível realizar o reparo automaticamente. B-Refactoring é capaz de dividir um

teste em múltiplos testes, de modo que cada teste cubra apenas um caminho de execução da aplicação. Vale destacar que o objetivo da abordagem é realizar uma refatoração sob demanda, isto é, os testes são refatorados antes de iniciar a execução da ferramenta de análise dinâmica e são revertidos para o código original após o término da execução. Sendo assim, o código de teste não é substituído permanentemente.

Hauptmann *et al.* (2015) tem como proposta a refatoração de casos de teste escritos em linguagem natural. Para identificar as similaridades entre os casos de teste a ferramenta utiliza técnicas de detecção de clones. A abordagem proposta decompõe os casos de teste de modo que as partes similares entre eles são extraídas e posteriormente reusadas por cada caso de teste. Essa extração é análoga ao que ocorre quando o código de teste é refatorado através da configuração delegada. A abordagem é especialmente útil para testes de sistema. Por exemplo, considerando testes de aceitação para uma aplicação *Web*, é comum que diversos desses casos de teste tenham como preparação a realização da autenticação. A abordagem é capaz de identificar que a autenticação ocorre em diversos casos de teste e é capaz, também, de extrair o trecho responsável pela autenticação para que ele seja reusado pelos testes. A abordagem apresentada é semiautomática, uma vez que a ferramenta refatora os casos de teste escritos em linguagem natural, porém o desenvolvedor ainda precisa alterar os respectivos métodos de teste, implementando a refatoração manualmente.

Lambiase *et al.* (2020) apresenta um plugin para o ambiente de desenvolvimento integrado (*Integrated Development Environment – IDE*) IntelliJ. O plugin, denominado Dart, é capaz de refatorar três fedores de teste conhecidos, sendo eles, *general fixture*, *eager test* e *lack of cohesion of test methods*. O primeiro ocorre quando os acessórios de uma classe de teste são muito gerais e os métodos de teste da classe usam apenas parte deles. Ao encontrar um *general fixture smell*, Dart extrai o teste em questão para uma nova classe de teste que conterá em seu método de configuração implícita apenas os acessórios necessários para o teste. Já o *eager test smell* ocorre quando um método de teste executa mais de um método da classe de produção em teste. O fedor aumenta a responsabilidade do teste, o que resulta em métodos de teste maiores e mais complexos. Além disso, é mais difícil encontrar a causa quando testes com esse fedor falham. Para esses casos, a ferramenta divide o método de teste em dois, de modo que cada método de teste irá

verificar isoladamente um método da classe de produção em teste. Por fim, *lack of cohesion of test methods* ocorre quando uma classe de teste contém métodos de teste que exercitam diferentes áreas do código de teste. Diz-se que os métodos de teste não possuem coesão. Como proposta de refatoração a ferramenta extrai os métodos de teste para classes separadas. Para identificar os fedores de teste, Dart utiliza uma segunda ferramenta, chamada Taste. Para realizar a refatoração, a ferramenta apresenta um aviso após a criação de um *commit*. O aviso mostra os fedores de código de teste encontrados, e o desenvolvedor tem, então, a opção de solicitar que a ferramenta realize automaticamente a refatoração.

Assim como Lambiase *et al.* (2020), Pizzini, Reinhehr and Mulucelli (2022, 2023) também apresentam um método para refatoração automática de *eager test smell*. A solução para remoção de tal fedor é a mesma apresentada por Lambiase *et al.* (2020). Isto é, o método de teste é dividido em dois ou mais métodos de teste de modo que cada método do objeto sendo testado terá seu próprio teste. O principal desafio apontado pelos autores é a correta identificação do fedor de teste. Isso porque um determinado método do objeto sendo testado pode ser utilizado como preparação do teste e não necessariamente como exercício. Nesse sentido, os autores consideram que o teste apresenta o fedor de teste apenas quando métodos de invocação do SUT são intercalados com as asserções para verificar o comportamento. Sendo assim, se o método de teste contém duas ou mais seções de verificação separadas por métodos de invocação do SUT, então considera-se que o teste em questão contém um *eager test smell*. Considerando um total de 350 classes de teste, sendo 312 delas afetadas por *eager test smell*, a abordagem conseguiu refatorar 295 classes sem erros. As demais 17 classes refatoradas apresentaram, no entanto, erros ou falhas. Destaca-se que a abordagem depende da ordenação dos métodos de teste refatorados para que possam ser executados com sucesso. Isso porque ela divide os testes e faz com esse o teste seguinte reusa a configuração de acessórios do teste anterior, evitando duplicação. Entretanto, essa característica faz com que os métodos de teste não sejam autocontidos, violando, assim, o princípio da independência defendido por Meszaros (2007). O tempo de execução entre a versão não refatorada e a versão refatorada aumentou em 23%. Os autores justificam que esse aumento foi decorrente do aumento do número de casos de teste, uma vez que não houve um aumento na quantidade de código de teste.

4. PROPOSTA

Duas atividades são necessárias na refatoração manual do código de teste. Primeiro, identificar duplicações de código de teste que podem ser removidas através do processo de refatoração. Identificar duplicação de código entre dois testes apenas exigirá um esforço proporcional à quantidade de código existente em cada teste. Nesse caso, o esforço é geralmente baixo. Entretanto, esse esforço não é linear conforme a quantidade de testes aumenta. Isso porque será necessário verificar cada par de testes. Assim, podemos dizer que o esforço para identificar testes com código comum cresce, *a priori*, em proporção fatorial¹¹ conforme o número de testes. A segunda atividade é realizar manualmente o próprio processo de refatoração. Para isso, é necessário modificar o código de teste adaptando-o para a nova estrutura. O problema no processo manual, além de requerer um maior esforço do desenvolvedor, é a possibilidade de inclusão de erros. Assim, após a refatoração é possível que o código de teste não sofra tão somente mudanças estruturais, mas também mudanças comportamentais. Quando o código da aplicação é refatorado, utiliza-se a execução dos testes para garantir que mudanças comportamentais não foram introduzidas. Entretanto, quando se considera refatoração do código de teste, não há como usar a execução dos testes como garantia de que mudanças comportamentais não foram introduzidas, uma vez que o processo de refatoração dos testes pode causar falsos positivos, isto é, o teste passar mesmo estando incorreto.

A Figura 8 apresenta um exemplo motivador envolvendo dois testes com duplicação de código. As linhas 4 a 10 do teste `sacarValorDoSaldo` são exatamente iguais as linhas 18 a 24 do teste `sacarMaisQueOSaldo`. Tal duplicação de código poderia ser removida através da estratégia de configuração implícita, conforme apresentado na Figura 9. A refatoração dos testes apresentados na Figura 8 através da estratégia de configuração implícita é um procedimento trivial que pode ser facilmente realizado de forma manual. Entretanto, é importante destacar que este trata-se de um exemplo simplificado para facilitar a compreensão. Quando um contexto maior é considerado, com centenas e até milhares de testes, por exemplo,

¹¹ A quantidade de combinações sem repetição par a par de n elementos é dada pela equação: $C_2^n = \binom{n}{2} = \frac{n!}{2!(n-2)!}$

fica mais fácil perceber que o processo manual de refatoração pode demandar alto esforço de desenvolvimento, podendo, inclusive, superar o ganho obtido com a melhoria da manutenibilidade devido à redução de duplicação de código de teste.

Figura 8. Classe de teste com duplicação de código.

```
1 class TesteSistemaBancarioComDuplicacao {
2     @Test
3     fun sacarValorDoSaldo() {
4         val casaDaMoeda = CasaDaMoeda(Moeda.BRL)
5         val sistemaBancario = SistemaBancario()
6         val bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
7         val trindade = bb.criarAgencia("Trindade")
8         val maria = trindade.criarConta("Maria", "000.000.000-00")
9         val cincoReais = casaDaMoeda.emitir(5)
10        maria.depositar(cincoReais)
11        val saque = maria.sacar(cincoReais)
12        assertTrue(saque.sucesso())
13        assertEquals(casaDaMoeda.zero(), maria.calcularSaldo())
14    }
15
16    @Test
17    fun sacarMaisQue0Saldo() {
18        val casaDaMoeda = CasaDaMoeda(Moeda.BRL)
19        val sistemaBancario = SistemaBancario()
20        val bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
21        val trindade = bb.criarAgencia("Trindade")
22        val maria = trindade.criarConta("Maria", "000.000.000-00")
23        val cincoReais = casaDaMoeda.emitir(5)
24        maria.depositar(cincoReais)
25        val dezReais = casaDaMoeda.emitir(10)
26        val saque = maria.sacar(dezReais)
27        assertFalse(saque.sucesso())
28        assertEquals(cincoReais, maria.calcularSaldo())
29    }
30 }
```

Figura 9. Classe de teste sem duplicação de código.

```

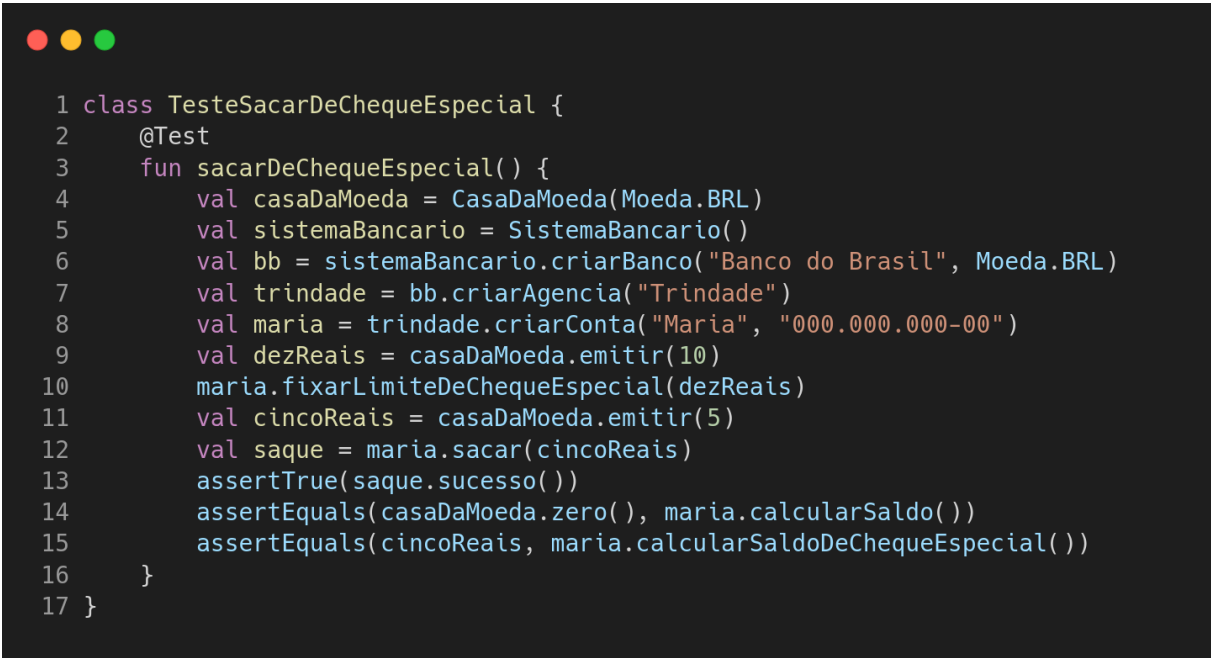
1 class TesteSistemaBancarioSemDuplicacao {
2     private lateinit var casaDaMoeda: CasaDaMoeda
3     private lateinit var cincoReais: Dinheiro
4     private lateinit var maria: Conta
5
6     @BeforeEach
7     fun configurar() {
8         casaDaMoeda = CasaDaMoeda(Moeda.BRL)
9         val sistemaBancario = SistemaBancario()
10        val bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
11        val trindade = bb.criarAgencia("Trindade")
12        maria = trindade.criarConta("Maria", "000.000.000-00")
13        cincoReais = casaDaMoeda.emitir(5)
14        maria.depositar(cincoReais)
15    }
16
17    @Test
18    fun sacarValorDoSaldo() {
19        val saque = maria.sacar(cincoReais)
20        assertTrue(saque.sucesso())
21        assertEquals(casaDaMoeda.zero(), maria.calcularSaldo())
22    }
23
24    @Test
25    fun sacarMaisQue0Saldo() {
26        val dezReais = casaDaMoeda.emitir(10)
27        val saque = maria.sacar(dezReais)
28        assertFalse(saque.sucesso())
29        assertEquals(cincoReais, maria.calcularSaldo())
30    }
31 }

```

Conforme o número de testes aumenta, torna-se mais difícil remover toda a duplicação de código através do processo manual de refatoração. Identificar se uma dada refatoração melhorará ou não a manutenibilidade dos testes pode ser um processo custoso. Por exemplo, seria vantajoso refatorar o teste apresentado na Figura 10, movendo-o para a classe apresentada na Figura 9, tendo como finalidade evitar a duplicação de código das linhas 4 a 8? Para responder esta pergunta, é preciso ter em mente que as linhas 13 a 14 da Figura 9 precisariam ser removidas do método de configuração implícita, `configurar`, uma vez que no teste da Figura 10, a operação de depósito não pode ocorrer. Com isso, as linhas supracitadas precisariam ser duplicadas entre os testes `sacarValorDoSaldo` e

`sacarMaisQueOSaldo`. Apesar disso, pode-se responder que sim, já que ainda seria possível reusar as linhas 8 a 12 da Figura 9 em mais um teste. Ao duplicar duas linhas, cinco linhas do código do teste da Figura 10 seriam removidas. Mas e se a classe da Figura 9 tivesse outros testes, ainda seria vantajoso? A resposta para essa pergunta depende da quantidade de testes existentes na classe. Se a classe já tivesse quatro testes, ao invés de apenas dois, então mover o teste `sacarDeChequeEspecial` para a classe já não valeria mais a pena, uma vez que para evitar a duplicação de cinco linhas, seis linhas seriam adicionadas.

Figura 10. Teste com duplicação de código em outra classe de teste.



```

1 class TesteSacarDeChequeEspecial {
2     @Test
3     fun sacarDeChequeEspecial() {
4         val casaDaMoeda = CasaDaMoeda(Moeda.BRL)
5         val sistemaBancario = SistemaBancario()
6         val bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
7         val trindade = bb.criarAgencia("Trindade")
8         val maria = trindade.criarConta("Maria", "000.000.000-00")
9         val dezReais = casaDaMoeda.emitir(10)
10        maria.fixarLimiteDeChequeEspecial(dezReais)
11        val cincoReais = casaDaMoeda.emitir(5)
12        val saque = maria.sacar(cincoReais)
13        assertTrue(saque.sucesso())
14        assertEquals(casaDaMoeda.zero(), maria.calcularSaldo())
15        assertEquals(cincoReais, maria.calcularSaldoDeChequeEspecial())
16    }
17 }

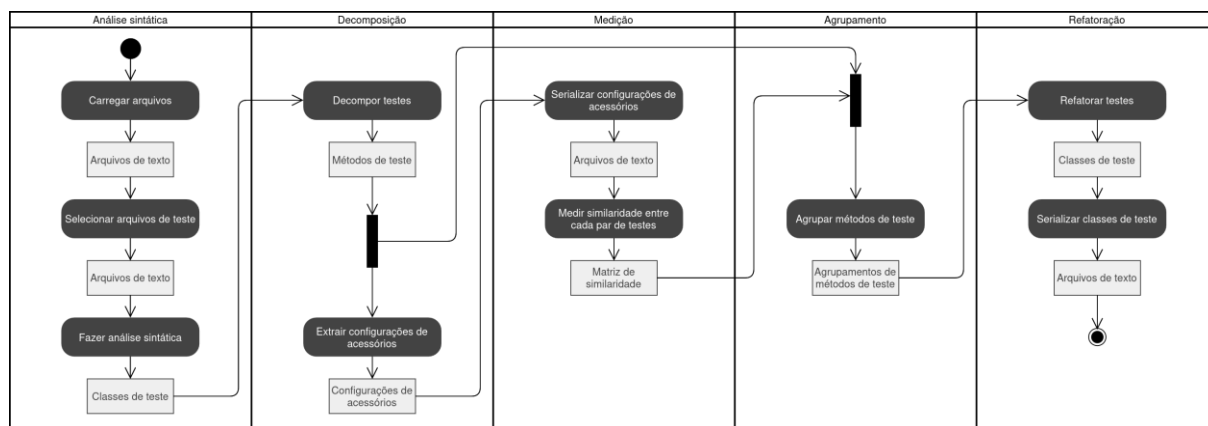
```

O objetivo do exemplo envolvendo a classe de teste da Figura 9 e o teste da Figura 10, é mostrar como o processo de refatoração manual através da estratégia de configuração implícita pode se tornar complicado conforme o número de testes aumenta. Escolher manualmente a melhor classe para adicionar um novo teste ou ainda encontrar a melhor distribuição de classes de teste, pode demandar grande esforço de desenvolvimento que nem sempre garantirá uma redução da duplicação de código. Para facilitar o processo de refatoração de código de teste e impedir que erros sejam introduzidos no código durante o processo de refatoração, este trabalho propõe a refatoração automática de classes de teste através do uso de métricas de similaridade e de algoritmos de agrupamento. Para isso, este trabalho propõe um

modelo para refatoração automática de classes de teste. A proposta tem como base a utilização de métricas de similaridade e algoritmos de agrupamento. O objetivo da utilização das métricas de similaridade é identificar pares de teste com código comum ou duplicado. Já o uso de algoritmos de agrupamentos tem como propósito maximizar o reuso de código através de um rearranjo na distribuição dos testes.

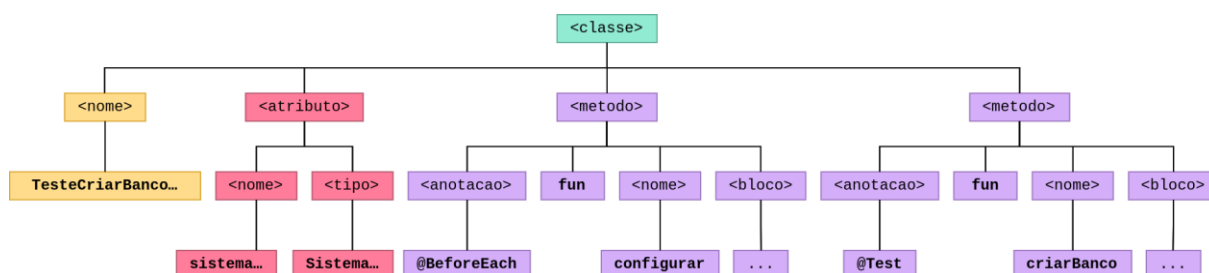
O modelo para refatoração automática de classes de teste é apresentado através do diagrama de atividades da UML da Figura 11. As atividades são divididas em 5 etapas. Primeiro é realizada uma análise sintática do código com objetivo de obter as informações necessárias para as etapas seguintes. Na sequência realizamos a etapa que chamamos de decomposição que visa obter para cada teste todo código de configuração de acessórios utilizado pelo teste, mesmo o código de configuração que está definido fora do método de teste. A configuração de acessórios de cada teste é utilizada na etapa seguinte, a medição. **É nesta etapa que a similaridade entre cada par de testes é avaliada.** Ao final, é gerada uma matriz de similaridade contendo o grau de similaridade de cada teste. A matriz de similaridade é utilizada pela etapa seguinte que agrupa os testes em classes tendo como objetivo maximizar o reuso de código através da estratégia de configuração implícita. A etapa final consiste em reconstruir o código de teste, criando uma classe de teste para cada agrupamento. Além disso, para cada agrupamento, deve-se aplicar uma refatoração automática, que moverá para o método de configuração implícita o código comum aos testes pertencentes à classe. A atividade final consiste em serializar cada classe de teste para o sistema de arquivos.

Figura 11. Diagrama de atividades do modelo para refatoração automática.



A primeira etapa consiste na análise sintática. Nela, os arquivos do projeto que terá suas classes de teste refatoradas são carregados e, posteriormente, filtrados. O filtro elimina arquivos do código de aplicação e outros arquivos que não sejam relativos ao código de teste. A etapa segue através da análise sintática do código de teste. Cada classe de teste identificada é representada através de uma AST. A classe de teste apresentada na Figura 5, por exemplo, é representada através da AST apresentada na Figura 12. Cada nó da árvore corresponde a um elemento da estrutura sintática da classe de teste. Por exemplo, o elemento raiz `<classe>` corresponde à própria classe, e seus nós filhos correspondem aos elementos que fazem parte da classe, como atributos e métodos. Na Figura 12, os elementos filhos de `<bloco>` foram omitidos por questões de simplificação. Eles representam as sentenças no código dos métodos de teste e de configuração. A transformação do código de teste em ASTs visa facilitar o processo de extração das informações na etapa seguinte.

Figura 12. Árvore abstrata de sintaxe de uma classe de teste.



Na etapa seguinte, decomposição, cada AST é percorrida com o objetivo de extrair as informações que serão necessárias para o restante do processo. A primeira atividade consiste em extrair o nome, a configuração de acessórios e as asserções de cada teste. Considerando o exemplo da Figura 12, para extrair as informações do teste, a AST será percorrida em busca de nós marcados com `<metodo>`. Ao encontrar, verifica-se se a subárvore possui algum nó marcado com `<anotacao>` e se tal nodo possui `@Teste` como nó folha. Caso as condições sejam satisfeitas, um teste foi encontrado. Assim, o restante da subárvore identificada como método de teste será percorrida para que sejam coletados o nome, as asserções e parte da configuração de acessórios. Destaca-se que apenas parte da configuração é coletada nesse momento, pois nem toda a configuração de

acessórios utilizada pelo teste será encontrada diretamente no método de teste. Por exemplo, na estratégia de configuração implícita, apresentada na Figura 5, parte da configuração de acessórios do teste está definida no método `configurar`, isto é, separado do método teste. Assim, para coletar o restante da configuração de acessórios utilizada pelo teste, realiza-se um processo análogo ao utilizado para encontrar o método de teste em si. Porém, agora procurando no nó `<anotacao>` por `@BeforeEach` ao invés de `@Teste`. Vale destacar que a decomposição realiza um processo inverso ao processo de refatoração. Nesse sentido, mesmo que dois testes reusam o mesmo código através de um método de configuração implícita da classe de teste, o processo de decomposição irá duplicar o código reusado. Isso é feito porque o objetivo é redistribuir os testes quando necessário. Assim, para que a etapa seguinte possa se eficiente na identificação de testes com código comum, toda a configuração de acessórios usada para o teste deve ser incluída. Além disso a decomposição não incluirá as asserções, uma vez que não está no escopo deste trabalho a fatoração de asserções por não ser um procedimento recomendado (Beck, 2003).

A etapa de medição consiste em avaliar a similaridade entre cada par de testes. Para isso, primeiro serializa-se a configuração de acessórios de cada teste para um arquivo separado. A serialização é realizada pois as métricas de similaridade terão como entrada os arquivos de código, e não os objetos em memória. Assim, para cada par de testes é gerado um grau de similaridade a partir da configuração de acessórios de cada teste. Esses graus de similaridade alimentarão, então, a matriz de similaridade. A Figura 13 apresenta um exemplo de matriz de similaridade considerando os testes da Figura 8 e o teste da Figura 10. Através da matriz, é possível obter a similaridade entre qualquer par de testes. Por exemplo, para obter a similaridade entre os testes `sacarValorDoSaldo` e `sacarMaisQueOSaldo` (0.9), basta obter o valor da cédula localizada pela primeira linha e pela segunda coluna da matriz.

Figura 13. Matriz de similaridade de testes.

	sacarValorDoSaldo	sacarMaisQueOSaldo	sacarDeChequeEspecial
sacarValorDoSaldo	1	0.9	0.7
sacarMaisQueOSaldo	0.9	1	0.8
sacarDeChequeEspecial	0.7	0.7	1

A geração da matriz de similaridade permite obter, para cada teste, uma classificação dos testes mais similares. A próxima etapa no processo de refatoração consiste em utilizar esta informação para agrupar em uma mesma classe de teste os testes que são similares entre si. Para isso, este trabalho propõe a utilização de algoritmos de agrupamento que utilizem o grau de similaridade entre cada par de teste para a criação dos grupos. O objetivo da etapa de agrupamento é maximizar a quantidade de código reusado. Nesse sentido, o algoritmo de agrupamento será tão mais eficiente do ponto de vista deste trabalho quanto maior for a quantidade de código que puder ser reusado.

A Figura 14 apresenta um exemplo de grupos de classes considerando os testes da Figura 8 e o teste da Figura 10. No exemplo, foram gerados dois grupos, um contendo os testes `sacarValorDoSaldo` e `sacarMaisQueOSaldo` e outro contendo apenas o teste `sacarDeChequeEspecial`. Cada grupo corresponde a uma classe de teste que deverá ser gerada no processo de refatoração. No entanto, os grupos apresentados na Figura 14 não correspondem à melhor opção de refatoração, uma vez que, conforme discutido no início desta seção, a opção que promoveria o maior reuso de código, considerando apenas os três testes, consiste em colocar o teste `sacarDeChequeEspecial` na mesma classe de teste que os testes `sacarValorDoSaldo` e `sacarMaisQueOSaldo`. Esse cenário é apresentado na Figura 15, onde um único grupo para todos os testes é gerado.

Figura 14. Dois grupos para três testes.

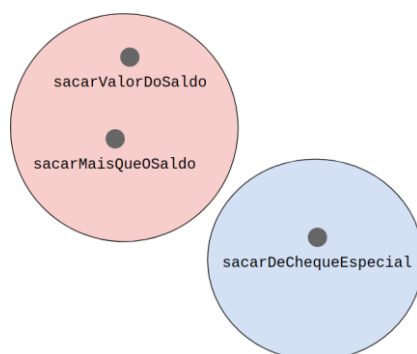
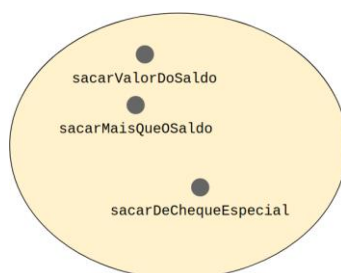


Figura 15. Um grupo para três testes.



Uma vez que os grupos de teste estejam definidos, a próxima e última etapa consiste em gerar as novas classes de teste. A etapa de refatoração deverá criar uma classe de teste para cada grupo. O código comum aos testes de cada grupo deverá ser reutilizado através da estratégia de configuração implícita. Por fim, as novas classes de teste são serializadas para o sistema de arquivos.

Como próximas etapas desse trabalho pretende-se iniciar a construção de um *framework* para o modelo proposto. O *framework* deve ser genérico e flexível, permitindo que novas linguagens, métricas de similaridade, algoritmos de agrupamento e algoritmos de refatoração possam ser incluídos. Inicialmente, o *framework* deverá suportar testes escritos na linguagem Java e a refatoração automática através da estratégia de configuração implícita. Além disso, pretende-se incluir no *framework* métricas de similaridade já existentes. A partir dessas métricas, iremos realizar um experimento para avaliar a efetividade de cada uma em identificar pares de teste que são bons candidatos para refatoração através da estratégia de configuração implícita. Além das métricas já existentes, iremos avaliar a possibilidade e necessidade de criar uma nova métrica, projetada especificamente para o contexto de teste e de refatoração através da configuração implícita. A etapa seguinte será implementar algoritmos de agrupamento no *framework*, bem como

realizar um experimento para avaliar quais algoritmos e configurações maximizam a quantidade de reuso ao serem usados para distribuir testes em classes de teste. Para a realização desse último experimento, também iremos implementar o algoritmo de refatoração em si.



5. CONCLUSÃO

A duplicação do código de teste é um problema que afeta a manutenibilidade dos testes a médio e longo prazo. Este trabalho apresentou as principais formas de reuso de código de teste encontradas na literatura. O reuso é alcançado através de estratégias de configuração de acessórios que promovem o compartilhamento do código. Evitar ou remover a duplicação do código de teste, entretanto, nem sempre é uma tarefa trivial, principalmente conforme o número de testes aumenta. Com o objetivo de facilitar o processo de refatoração de código de teste e reduzir, portanto, o esforço de desenvolvimento e manutenção dos testes, este trabalho propõe uma abordagem para refatoração automática do código de teste. A abordagem emprega métricas de similaridade e algoritmos de agrupamento para automatizar a refatoração do código de teste. Primeiramente, aplicam-se métricas de similaridade para identificar pares de testes similares. Posteriormente, utilizam-se algoritmos de agrupamento para definir conjuntos de testes similares. Cada grupo resultante origina uma classe de teste, onde o código comum é refatorado por meio da estratégia de configuração implícita.

Através de um estudo sistemático foi observado que a proposta possui potencial para contribuir com estado da arte, já que traz uma abordagem que não foi encontrada na literatura atual. Por isso, como sequência do trabalho, iremos implementar o modelo proposto através em um *framework*. Esse *framework* deve ser flexível para permitir diferentes combinações de métricas de similaridade, algoritmos de agrupamento e algoritmos de refatoração. Inicialmente, o foco será estudar as métricas de similaridade buscando suportar diferentes métricas no *framework* a ser desenvolvido. O objetivo é identificar quais métricas de similaridade de código são mais eficientes para identificar pares de teste com alto potencial de reuso. Na sequência faremos o mesmo para os algoritmos de agrupamento. Em especial queremos descobrir meios de identificar quando não é mais vantajoso agrupar um novo teste na classe. Por fim, iremos implementar a etapa de refatoração através do uso da estratégia de configuração implícita. Avaliaremos também, outras possíveis formas de fatoração.

REFERÊNCIAS

- ALVESTAD, Kristian. **Domain Specific Languages for Executable Specifications**. 2007. Dissertação de Mestrado. Institutt for datateknikk og informasjonsvitenskap.
- BAEZA-YATES, Ricardo; RIBEIRO-NETO, Berthier. **Modern information retrieval**. New York: ACM press, 1999.
- BAXTER, Ira D. et al. **Clone detection using abstract syntax trees**. In: Proceedings. International Conference on Software Maintenance (Cat. No. 98CB36272). IEEE, 1998. p. 368-377.
- BECK, Kent. **Test-driven development: by example**. Addison-Wesley Professional, 2003.
- BECK, Kent; GAMMA, Erich. **Extreme programming explained: embrace change**. addison-wesley professional, 2000.
- BEIZER, Boris. **Software Testing Techniques**. 2. ed. Van Nostrand Reinhold Company. 1990.
- BERGHEL, Hal L.; SALLACH, David L. **Measurements of program similarity in identical task environments**. ACM SIGPLAN Notices, v. 19, n. 8, p. 65-76, 1984.
- BERNARD, E. et al. **Tool Support for Refactoring Manual Tests**. HAL (Le Centre pour la Communication Scientifique Directe), v. 20, p. 332–342, 5 ago. 2020.
- BERNER, Stefan; WEBER, Roland; KELLER, Rudolf K. **Observations and lessons learned from automated testing**. In: Proceedings of the 27th international conference on Software engineering. ACM, 2005. p. 571-579.
- BERTOLINO, Antonia. **Software testing research: Achievements, challenges, dreams**. In: 2007 Future of Software Engineering. IEEE Computer Society, 2007. p. 85-103.
- BIRANT, Derya; KUT, Alp. **ST-DBSCAN: An algorithm for clustering spatial-temporal data**. Data & Knowledge Engineering, v. 60, n. 1, p. 208-221, 2007.
- BOURQUE, Pierre et al. **Guide to the software engineering body of knowledge (SWEBOK (R))**: Version 3.0. IEEE Computer Society Press, 2014.

CAO, Qiong; YING, Yiming; LI, Peng. **Similarity metric learning for face recognition**. In: Proceedings of the IEEE International Conference on Computer Vision. 2013. p. 2408-2415.

CHRISTENSEN, Claus A. et al. **A unit-test framework for database applications**. In: 2006 10th International Database Engineering and Applications Symposium (IDEAS'06). IEEE, 2006. p. 11-20.

CHVÁTAL, Vašek; KLARNER, David A.; KNUTH, Donald Ervin. **Selected combinatorial research problems**. Computer Science Department, Stanford University, 1972.

DALAL, Siddhartha R. et al. **Model-based testing in practice**. In: Proceedings of the 1999 International Conference on Software Engineering (IEEE Cat. No. 99CB37002). IEEE, 1999. p. 285-294.

DONALDSON, John L.; LANCASTER, Ann-Marie; SPOSATO, Paula H. **A plagiarism detection system**. In: ACM SIGCSE Bulletin. ACM, 1981. p. 21-25.

DORNELES, Carina F. et al. **Measuring similarity between collection of values**. In: Proceedings of the 6th annual ACM international workshop on Web information and data management. ACM, 2004. p. 56-63.

DUCASSE, Stéphane; RIEGER, Matthias; DEMEYER, Serge. **A language independent approach for detecting duplicated code**. In: Proceedings IEEE International Conference on Software Maintenance-1999 (ICSM'99). 'Software Maintenance for Business Change' (Cat. No. 99CB36360). IEEE, 1999. p. 109-118.

EMERY, Dale H. **Writing maintainable automated acceptance tests**. In: Agile Testing Workshop, Agile Development Practices, Orlando, Florida. 2009.

FAIDHI, Jinan AW; ROBINSON, Stuart K. **An empirical approach for detecting program similarity and plagiarism within a university programming environment**. Computers & Education, v. 11, n. 1, p. 11-19, 1987.

FERRANTE, Jeanne; OTTENSTEIN, Karl J.; WARREN, Joe D. **The program dependence graph and its use in optimization**. ACM Transactions on Programming Languages and Systems (TOPLAS), v. 9, n. 3, p. 319-349, 1987.

FOWLER, Martin. **Refactoring: improving the design of existing code**. Pearson Education India, 1999.

FREEMAN, Steve; PRYCE, Nat. **Growing object-oriented software, guided by tests**. Pearson Education, 2009.

GREILER, Michaela et al. **Strategies for avoiding text fixture smells during software evolution**. In: Proceedings of the 10th Working Conference on Mining Software Repositories. IEEE Press, 2013. p. 387-396.

GREILER, Michaela; VAN DEURSEN, Arie; STOREY, Margaret-Anne. **Automated detection of test fixture strategies and smells**. In: 2013 IEEE Sixth International Conference on Software Testing, Verification and Validation. IEEE, 2013. p. 322-331.

GRIER, Sam. **A tool that detects plagiarism in Pascal programs**. In: ACM SIGCSE Bulletin. Acm, 1981. p. 15-20.

HAMMING, Richard W. **Error detecting and error correcting codes**. The Bell system technical journal, v. 29, n. 2, p. 147-160, 1950.

HARROLD, M. Jean; GUPTA, Rajiv; SOFFA, Mary Lou. **A methodology for controlling the size of a test suite**. ACM Transactions on Software Engineering and Methodology (TOSEM), v. 2, n. 3, p. 270-285, 1993.

HARTIGAN, John A.; WONG, Manchek A. **Algorithm AS 136**: A k-means clustering algorithm. Journal of the Royal Statistical Society. Series C (Applied Statistics), v. 28, n. 1, p. 100-108, 1979.

HAUGSET, Børge; HANSSEN, Geir Kjetil. **Automated acceptance testing**: A literature review and an industrial case study. In: Agile 2008 Conference. IEEE, 2008. p. 27-38.

HAUPTMANN, B. et al. **Generating Refactoring Proposals to Remove Clones from Automated System Tests**. In: 2015 IEEE 23rd International Conference on Program Comprehension, p. 115-124. IEEE, 2015.

HELFMAN, Jonathan. **Dotplot patterns**: a literal look at pattern languages. TAPoS, v. 2, n. 1, p. 31-41, 1996.

HEMMATI, Hadi; ARCURI, Andrea; BRIAND, Lionel. **Reducing the cost of model-based testing through test case diversity**. In: IFIP International Conference on Testing Software and Systems. Springer, Berlin, Heidelberg, 2010. p. 63-78.

HEMMATI, Hadi; BRIAND, Lionel. **An industrial investigation of similarity measures for model-based test case selection**. In: 2010 IEEE 21st International Symposium on Software Reliability Engineering. IEEE, 2010. p. 141-150.

HEMMATI, Hadi et al. **An enhanced test case selection approach for model-based testing: an industrial case study**. In: Proceedings of the eighteenth ACM SIGSOFT international symposium on Foundations of software engineering. ACM, 2010. p. 267-276.

HOLMES, Reid; MURPHY, Gail C. **Using structural context to recommend source code examples**. In: Proceedings. 27th International Conference on Software Engineering, 2005. ICSE 2005. IEEE, 2005. p. 117-125.

INDYK, Piotr; MOTWANI, Rajeev. **Approximate nearest neighbors: towards removing the curse of dimensionality**. In: Proceedings of the thirtieth annual ACM symposium on Theory of computing. ACM, 1998. p. 604-613.

JACCARD, Paul. **The distribution of the flora in the alpine zone**. 1. New phytologist, v. 11, n. 2, p. 37-50, 1912.

JAIN, Anil K.; MURTY, M. Narasimha; FLYNN, Patrick J. **Data clustering: a review**. ACM computing surveys (CSUR), v. 31, n. 3, p. 264-323, 1999.

JOHNSON, Richard Arnold et al. **Applied multivariate statistical analysis**. Upper Saddle River, NJ: Prentice hall, 2007.

KAMIYA, Toshihiro; KUSUMOTO, Shinji; INOUE, Katsuro. **CCFinder: a multilinguistic token-based code clone detection system for large scale source code**. IEEE Transactions on Software Engineering, v. 28, n. 7, p. 654-670, 2002.

KAPSER, Cory; GODFREY, Michael W. **Toward a taxonomy of clones in source code: A case study**. Evolution of large scale industrial software architectures, v. 16, p. 107-113, 2003.

KOMONDOOR, Raghavan; HORWITZ, Susan. **Using slicing to identify duplication in source code**. In: International static analysis symposium. Springer, Berlin, Heidelberg, 2001. p. 40-56.

KRINKE, Jens. **Identifying similar code with program dependence graphs**. In: Proceedings Eighth Working Conference on Reverse Engineering. IEEE, 2001. p. 301-309.

- LAMBIASE, S. et al. **Just-In-Time Test Smell Detection and Refactoring**. In: Proceedings of the 28th international conference on program comprehension, pp. 441-445. 2020.
- LANDHÄUßER, Mathias; TICHY, Walter F. **Automated test-case generation by cloning**. In: 2012 7th International Workshop on Automation of Software Test (AST). IEEE, 2012. p. 83-88.
- LEDRU, Yves et al. **Prioritizing test cases with string distances**. Automated Software Engineering, v. 19, n. 1, p. 65-95, 2012.
- LEVENSHTEIN, Vladimir I. **Binary codes capable of correcting deletions, insertions, and reversals**. In: Soviet physics doklady. 1966. p. 707-710.
- LONGO, Douglas Hiura et al. **Fixture Setup through Object Notation for Implicit Test Fixtures**. Journal of Computer Science, v. 11, n. 6, p. 794, 2015.
- MANBER, Udi et al. **Finding Similar Files in a Large File System**. In: Usenix Winter. 1994. p. 1-10.
- MATTSSON, Michael; BOSCH, Jan; FAYAD, Mohamed E. **Framework integration problems, causes, solutions**. Communications of the ACM, v. 42, n. 10, p. 80-87, 1999.
- MESZAROS, Gerard. **xUnit test patterns: Refactoring test code**. Pearson Education, 2007.
- MIRANDA, Breno et al. **Fast approaches to scalable similarity-based test case prioritization**. In: Proceedings of the 40th International Conference on Software Engineering. ACM, 2018. p. 222-232.
- NG, Raymond T.; HAN, Jiawei. **CLARANS: A method for clustering objects for spatial data mining**. IEEE Transactions on Knowledge & Data Engineering, n. 5, p. 1003-1016, 2002.
- OPDYKE, William F. **Refactoring object-oriented frameworks**. 1992.
- OTTENSTEIN, Karl J. **An algorithmic approach to the detection and prevention of plagiarism**. 1976.
- PIZZINI, A.; REINEHR, S.; ANDREIA MALUCELLI. **Automatic Refactoring Method to Remove Eager Test Smell**. In Proceedings of the XXI Brazilian Symposium on Software Quality, p. 1-10. 2022.

PIZZINI, A.; REINEHR, S.; ANDREIA MALUCELLI. **Sentinel: A process for automatic removing of Test Smells**. In Proceedings of the XXII Brazilian Symposium on Software Quality, pp. 80-89. 2023.

RAGKHITWETSAGUL, Chaoyong; KRINKE, Jens; CLARK, David. **A comparison of code similarity analysers**. Empirical Software Engineering, v. 23, n. 4, p. 2464-2519, 2018.

ROY, Chanchal K.; CORDY, James R. **NICAD: Accurate detection of near-miss intentional clones using flexible pretty-printing and code normalization**. In: 2008 16th IEEE international conference on program comprehension. IEEE, 2008. p. 172-181.

SANTANA, R. et al. **Refactoring Assertion Roulette and Duplicate Assert test smells: a controlled experiment**. arXiv (Cornell University), 1 jan. 2022.

SAHAMI, Mehran; HEILMAN, Timothy D. **A web-based kernel function for measuring the similarity of short text snippets**. In: Proceedings of the 15th international conference on World Wide Web. AcM, 2006. p. 377-386.

SHI, Y.-Q.; HUANG, S.; WAN, J.-Y. **A Reuse-oriented Clustering Method for Test Cases**. 2022 9th International Conference on Dependable Systems and Their Applications (DSA), p. 153–162, 1 ago. 2022.

SILVA, Lucas P.; VILAIN, Patricia. **Execution and code reuse between test classes**. In: 2016 IEEE 14th International Conference on Software Engineering Research, Management and Applications (SERA). IEEE, 2016. p. 99-106.

SILVA, Lucas P.; VILAIN, Patricia. **Reuse of Fixture Setup between Test Classes**. In: SEKE. 2017. p. 224-229.

SOBERNIG, Stefan; ZDUN, Uwe. **Inversion-of-control layer**. In: Proceedings of the 15th European Conference on Pattern Languages of Programs. ACM, 2010. p. 21.

SPADINI, D. et al. **To Mock or Not to Mock? An Empirical Study on Mocking Practices**. 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR), maio 2017.

SWEET, Richard E. **The Mesa programming environment**. In: ACM SIGPLAN Notices. ACM, 1985. p. 216-229.

TIWARI, Rajeev; GOEL, Noopur. **Reuse: reducing test effort**. ACM SIGSOFT Software Engineering Notes, v. 38, n. 2, p. 1-11, 2013.

TSAI, Wei-tek et al. **Scenario-based object-oriented testing framework**. In: Third International Conference on Quality Software, 2003. Proceedings. IEEE, 2003. p. 410-417.79

VAN DEURSEN, Arie et al. **Refactoring test code**. In: Proceedings of the 2nd international conference on extreme programming and flexible processes in software engineering (XP2001). 2001. p. 92-95.

VON LUXBURG, U. **A tutorial on spectral clustering**. *Statistics and Computing*, v. 17, n. 4, p. 395–416, 22 ago. 2007.

WANG, Wei et al. **STING: A statistical information grid approach to spatial data mining**. In: VLDB. 1997. p. 186-195.

WANG, X. et al. **An automatic refactoring framework for replacing test-production inheritance by mocking mechanism**. Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 18 ago. 2021.

WANG, X et al. **JMocker: refactoring test-production inheritance by mockito**. In: Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings, p. 125-129, 2022a.

WANG, X. et al. **From Inheritance to Mockito: An Automatic Refactoring Approach**. IEEE Transactions on Software Engineering, p. 1–23, 2022b.

WONG, W. Eric et al. **A study of effective regression testing in practice**. In: PROCEEDINGS The Eighth International Symposium On Software Reliability Engineering. IEEE, 1997. p. 264-274.

XUAN, J. et al. **B-Refactoring: Automatic test code refactoring to improve dynamic analysis**. Information and Software Technology, v. 76, p. 65–80, ago. 2016.